

Convolutional Neural Networks & Computer Vision

Vorlesungsskript · Woche 2

Roter Faden der Veranstaltung. Die Vorlesung folgt dem ML-Lifecycle: *Modelle bauen* → *Modelle produktiv machen* → *Modelle verantworten* → Capstone. In diesem Skript entsteht das **Trägermodell**: ein per Transfer Learning angepasstes Faltungsnetz zur Bildklassifikation, das im weiteren Semester versioniert, als Service deployt, auf Drift überwacht, erklärt und dokumentiert wird. Was Sie hier bauen, begleitet Sie bis zum Capstone.

Inhaltsverzeichnis

Vorbemerkung und Organisatorisches	5
1 Einführung: Warum Bilder eine eigene Architektur brauchen	6
1.1 Der Bruch mit dem MLP	6
1.2 Was CNNs verändert haben	6
1.3 Die drei Grundaufgaben der Computer Vision	7
1.4 Leitbeispiele	7
2 Die Faltungsoperation	7
2.1 Von der Einheit zum Filter	7
2.2 Drei Ideen hinter der Faltung	8
2.3 Padding und Stride	9
2.4 Kanäle: Mehrere Filter, mehrere Eingaben	9
2.5 Rezeptives Feld und Tiefe	10
2.6 Faltung in PyTorch	10
3 Pooling, Invarianz und das vollständige CNN	11
3.1 Pooling	11
3.2 Anatomie eines CNN	11
3.3 LeNet-5: Das erste CNN für MNIST	12
3.4 Datenaugmentierung für Bilder	13
4 Architekturen: Von LeNet zu ResNet	13
4.1 AlexNet und VGG: Tiefe durch Training ermöglichen	13
4.2 GoogLeNet: Rechenaufwand durch 1×1 -Faltungen zähmen	14
4.3 ResNet: Das Degradationsproblem	14
4.4 Residual Learning	15
4.5 Die ResNet-Familie	16
4.6 Seit ResNet	17
4.7 Eine Architektur auswählen	17
5 Transfer Learning: Das Trägermodell entsteht	17
5.1 Warum Vortraining übertragbar ist	18
5.2 Welche Schichten trainieren?	18
5.3 Rezept und Implementierung	19
5.4 Das Trägermodell: Festlegung	20
6 Objektdetektion	20
6.1 Bounding Boxes und Intersection over Union (IoU)	21
6.2 Der klassische Weg: Klassifikator plus Suche	21
6.3 YOLO: Detektion als Regression	22
7 Segmentierung (Überblick)	23
7.1 Aufgabe und Ausgabestruktur	23
7.2 Encoder-Decoder und Upsampling	24
7.3 U-Net	24

8	Fehlklassifikationen analysieren und ins Netz hineinsehen	25
8.1	Konfusionsmatrix und klassenweise Metriken	25
8.2	Beispiele ansehen	26
8.3	Was hat das Netz gelernt?	26

Vorbemerkung und Organisatorisches

Organisatorisches – Woche 2

Termin	KW 42 (15.10), 4-SWS-Block (Theorie) und 2-SWS-Block (Lab)
Block	A – Vertiefung Deep Learning
Pflichtlektüre	[1]Goodfellow et al. (2016), Kap. 9 (<i>Convolutional Networks</i>)
Vertiefung	[2]He et al. (2015): <i>Deep Residual Learning for Image Recognition</i> · [3]Redmon et al. (2016): <i>You Only Look Once: Unified, Real-Time Object Detection</i>
Alternativ	[4]Bishop & Bishop (2024), Kap. 10 (<i>Convolutional Networks</i> , inkl. 10.4 Object Detection und 10.5 Image Segmentation) · [5]Murphy (2022), <i>PML: An Introduction</i> , Kap. 14 (<i>Neural Networks for Images</i>) und Abschn. 19.2 (<i>Transfer Learning</i>)
Online	https://cs231n.github.io/convolutional-networks/ · https://distill.pub/2017/feature-visualization/
Lab	LeNet-artiges CNN auf MNIST; Transfer Learning mit ResNet-18 auf Imagenette-Teilmenge → Trägermodell
Fehleranalyse	

Notation

Die Notation aus dem Skript *Vom Perzeptron zum Deep Network* gilt weiter (fett = Vektor/Matrix, $\mathcal{L}$ = Verlust, ϵ = Lernrate). Neu in diesem Skript:

Symbol	Typ	Bedeutung
$I(j, k), \mathbf{I} \in \mathbb{R}^{J \times K}$	Bild	Graustufenbild mit J Zeilen und K Spalten; Farbbild: Tensor $J \times K \times C$ mit $C = 3$ Kanälen
$\mathbf{K} \in \mathbb{R}^{M \times M}$	Kernel / Filter	lernbare Gewichte einer Faltungseinheit; bei C Eingangskanälen $M \times M \times C$
$\mathbf{C} = \mathbf{I} * \mathbf{K}$	Feature Map	Ausgabe einer Faltung (in der ML-Literatur meist als Kreuzkorrelation definiert, Abschn. 2)
P, S	Skalar	Padding (Randauffüllung in Pixeln) und Stride (Schrittweite) einer Faltung
$C_{\text{in}}, C_{\text{out}}$	Skalar	Zahl der Eingangs- bzw. Ausgangskanäle einer Schicht
$\mathcal{F}(\mathbf{x}), \mathcal{H}(\mathbf{x})$	Funktion	Residualfunktion bzw. gewünschte Abbildung eines Residualblocks: $\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$
$\mathbf{b} = (b_x, b_y, b_w, b_h)$	Vektor	Bounding Box: Mittelpunkt, Breite, Höhe (relativ zum Bild oder in Pixeln)
$\text{IoU}(\mathbf{A}, \mathbf{B})$	Skalar $\in [0, 1]$	Intersection over Union zweier Boxen
θ_b, θ_h	Parameter	Parameter des vortrainierten <i>Backbone</i> und des neuen <i>Kopfes</i> beim Transfer Learning

Tensorreihenfolge: Mathematisch schreiben wir Höhe $\times$ Breite $\times$ Kanäle; PyTorch speichert Bilder als (N, C, H, W) – Batch, Kanäle, Höhe, Breite. Diese Reihenfolge ist die häufigste Fehlerquelle beim Einstieg.

1 Einführung: Warum Bilder eine eigene Architektur brauchen

1.1 Der Bruch mit dem MLP

Im Skript *Vom Perzeptron zum Deep Network* wurde jedes MNIST-Bild zu einem Vektor $\mathbf{x} \in \mathbb{R}^{784}$ abgeflacht. Das MLP hat damit gut gearbeitet (rund 98 % Testgenauigkeit) aber es hat dabei etwas Entscheidendes ignoriert: *welche* Pixel nebeneinanderliegen. Für ein vollverbundenes Netz ist Pixel (3, 4) von Pixel (3, 5) genauso weit entfernt wie von Pixel (27, 27). Würde man alle Pixel mit einer festen Permutation vertauschen und diese Permutation konsistent auf Trainings- und Testdaten anwenden, änderte sich die Leistung des MLP nicht [4, S. 287]. Ein Mensch dagegen könnte in dem permutierten Bild keine Ziffer mehr erkennen.

Das ist der Kern dieses Skripts: Bilder sind *strukturierte Daten*. Zwei Eigenschaften natürlicher Bilder sind so verlässlich, dass man sie nicht aus Daten lernen, sondern direkt in die Architektur einbauen sollte:

1. **Lokalität.** Benachbarte Pixel sind stark korreliert; ein bedeutungsvolles Merkmal (Kante, Ecke, Textur) ist in einem kleinen Bildausschnitt erkennbar.
2. **Translationsinvarianz der Statistik.** Ein Auge sieht oben links im Bild genauso aus wie unten rechts. Ein Detektor, der es an einer Stelle erkennt, sollte es überall erkennen, auch ohne Trainingsbeispiele an jeder möglichen Position.

Ein Netz, das diese beiden Annahmen respektiert, kommt mit dramatisch weniger Parametern und Trainingsdaten aus und generalisiert besser. Das *Convolutional Neural Network* (CNN) ist genau dieses Netz [1], [6]. Ein zweites Argument ist rein quantitativ (Abb. 1): Eine vollverbundene erste Schicht mit 1 000 Einheiten auf einem 1000×1000 -RGB-Bild hätte $3 \cdot 10^9$ Gewichte und wäre damit quasi untrainierbar sowie speichersprengend. Eine Faltungsschicht mit 64 Filtern der Größe 3×3 hat 1 792 Parameter, unabhängig von der Bildgröße.

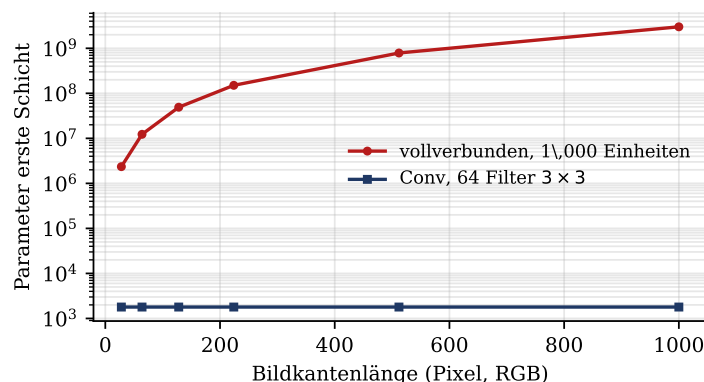


Abbildung 1: Parameterzahl der ersten Schicht in Abhängigkeit von der Bildgröße. Die vollverbundene Schicht wächst quadratisch mit der Kantenlänge; die Faltungsschicht ist konstant, weil ihre Gewichte über alle Positionen geteilt werden.

1.2 Was CNNs verändert haben

CNNs waren die erste tiefe Architektur mit industriellem Einsatz: LeNet-5 las ab Ende der 1990er Jahre Schecks und Postleitzahlen [6]. Der Durchbruch für die breite Wahrnehmung kam 2012, als AlexNet den ImageNet-Wettbewerb (1,28 Mio. Trainingsbilder, 1 000 Klassen) mit 15,3 % Top-5-Fehler gegenüber den rund 26 % des besten klassischen Verfahrens [7] gewann. Danach fiel der Fehler jedes Jahr: VGG (2014, 7,3 %), GoogLeNet (2014, 6,7 %), ResNet (2015, 3,6 % mit Ensemble) [2], [8], [9]. Menschen liegen auf demselben Datensatz bei etwa 5 % [4]. Seit 2020 sind Transformer-Architekturen (Vision Transformer, ViT) in einigen Aufgaben konkurrenzfähig oder überlegen; für kleine und mittlere Datensätze, auf Edge-Hardware und als Backbone für Detektion und Segmentierung sind CNNs weiterhin Standard und der Ausgangspunkt für alles, was im Skript *Sequenzmodelle & Transformer* folgt.

1.3 Die drei Grundaufgaben der Computer Vision

Dieses Skript behandelt drei Aufgaben mit steigendem Detailgrad der Ausgabe (Tab. 1). Alle drei nutzen dieselben Faltungsbausteine, unterscheiden sich jedoch im *Kopf* des Netzes und in der Verlustfunktion.

	Klassifikation	Objektdetektion	Segmentierung
Frage	Was ist im Bild?	Was ist wo im Bild?	Welches Pixel gehört wozu?
Ausgabe	eine Klassenverteilung pro Bild	Liste von Boxen ($\mathbf{b}$, Klasse, Konfidenz)	Klassenkarte $J \times K \times C$
Verlust	Cross-Entropy	Regression (Box) + Klassifikation + Objektheit	pixelweise Cross-Entropy
Beispiel	Hautläsion gutartig/bösartig	Fußgänger im Kamerabild	Tumorgewebe im MRT
In diesem Skript	Abschn. 2–5 (Trägermodell)	Abschn. 6 (YOLO)	Abschn. 7 (Überblick)

Tabelle 1: Die drei Grundaufgaben der Computer Vision und ihre Ausgabestruktur.

1.4 Leitbeispiele

Zwei Datensätze ziehen sich durch Skript und Lab. **MNIST** kennen Sie bereits und wird als Beispiel um zu zeigen, was ein kleines CNN gegenüber dem MLP gewinnt. **Imagenette howard_imagenette_2019** ist eine von fast.ai kuratierte Teilmenge von ImageNet mit zehn leicht unterscheidbaren Klassen (Kettensäge, Kirche, Kassettenrecorder, Golfball, Fallschirm, Englischer Springer, Französisches Horn, Müllwagen, Tankstelle, Fisch). Für das Trägermodell verwenden wir fünf dieser Klassen bei 160 px Auflösung mit $\approx 2\,500$ Trainings- und 500 Validierungsbildern. Der Datensatz ist absichtlich klein: Er zwingt uns zu Transfer Learning, und das Modell muss auf einem Laptop in Sekunden inferieren, damit es in den Skripten *MLOps: Vom Notebook zum Produktivsystem, Monitoring & Drift* und *Erklärbare KI & Compliance* weiterverwendbar bleibt.

2 Die Faltungsoperation

2.1 Von der Einheit zum Filter

Eine Einheit der ersten verdeckten Schicht eines MLP sieht das ganze Bild. Eine Faltungseinheit sieht nur einen kleinen Ausschnitt, etwa 3×3 Pixel, das sog. *rezeptive Feld*. Ihre Ausgabe ist wie gewohnt eine gewichtete Summe mit Nichtlinearität, $z = \text{ReLU}(\mathbf{w}^\top \mathbf{x} + w_0)$, nur dass $\mathbf{x}$ jetzt die neun Pixelwerte des Ausschnitts enthält. Die neun Gewichte lassen sich selbst als kleines 3×3 -Bild anordnen; man nennt sie *Kernel* oder *Filter* $\mathbf{K}$. Da $\mathbf{w}^\top \mathbf{x}$ bei fester Norm $\|\mathbf{x}\|$ genau dann maximal ist, wenn $\mathbf{x} \propto \mathbf{w}$, reagiert die Einheit am stärksten, wenn der Bildausschnitt *so aussieht wie der Kernel* [4]. Eine Faltungseinheit ist also ein Musterdetektor.

Der entscheidende Schritt ist nun, *denselben* Filter an jeder Position des Bildes anzuwenden. Die Ausgaben aller Positionen bilden zusammen die *Feature Map* $\mathbf{C}$.

Definition 2.1: Faltung (Kreuzkorrelation) einer Feature Map

Für ein Bild $\mathbf{I} \in \mathbb{R}^{J \times K}$ und einen Kernel $\mathbf{K} \in \mathbb{R}^{M \times M}$ ist die Feature Map

$$C(j, k) = \sum_{l=0}^{M-1} \sum_{m=0}^{M-1} I(j+l, k+m) K(l, m), \quad (1)$$

kurz $\mathbf{C} = \mathbf{I} * \mathbf{K}$; Bias und Nichtlinearität folgen wie gewohnt. Streng genommen ist Gl. (1) eine *Kreuzkorrelation*; die mathematische Faltung verwendet $I(j-l, k-m)$, also den gespiegelten Kernel. Da der Kernel gelernt wird, ist die Spiegelung ohne Bedeutung $\Rightarrow$ ein Netz, welches $\mathbf{K}$ lernt, lernt genauso gut das gespiegelte $\mathbf{K}$. Die ML-Literatur und alle Frameworks (nn.Conv2d) nennen Gl. (1) *Faltung* [1].

$$\begin{array}{|c|c|c|c|} \hline 2 & 5 & -3 & 0 \\ \hline 0 & 6 & 0 & -4 \\ \hline -1 & -3 & 0 & 2 \\ \hline 5 & 0 & 0 & 3 \\ \hline \end{array} * \begin{array}{|c|c|} \hline -2 & 0 \\ \hline 4 & 6 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 32 & 14 & -18 \\ \hline -22 & -24 & 12 \\ \hline 22 & 6 & 18 \\ \hline \end{array}$$

$I (4 \times 4) \qquad K (2 \times 2) \qquad C (3 \times 3)$
 $C(0,0) = 2 \cdot (-2) + 5 \cdot 0 + 0 \cdot 4 + 6 \cdot 6 = 32$

Abbildung 2: Faltung eines 4×4 -Bildes mit einem 2×2 -Kernel ohne Padding (Stride 1). Jede Ausgabezelle ist das Skalarprodukt des Kernels mit dem Bildausschnitt gleicher Größe; der Kernel wird dabei über alle 3×3 Positionen geschoben. (Rechnung nach [4, Abb. 10.3])

Beispiel 2.2: Handgebaute Kantendetektoren

Bevor Filter gelernt wurden, hat man sie manuell entworfen. Der Filter

$$K_{\text{vert}} = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

misst die Intensitätsänderung von links nach rechts. Seine Antwort ist groß, wo eine vertikale Kante liegt, und null auf homogenen Flächen. Der transponierte Filter $K_{\text{hor}} = K_{\text{vert}}^T$ findet entsprechend horizontale Kanten. Abbildung 3 zeigt beide Antworten auf einem synthetischen Bild: hell/dunkel codiert das Vorzeichen (Übergang dunkel→hell oder umgekehrt). Ein CNN lernt in der ersten Schicht Filter, die diesen handgebauten sehr ähnlich sehen (Abschn. 8.3). Zusätzlich lernt es, welche Kombinationen der Filter für die Aufgabe nützlich sind.

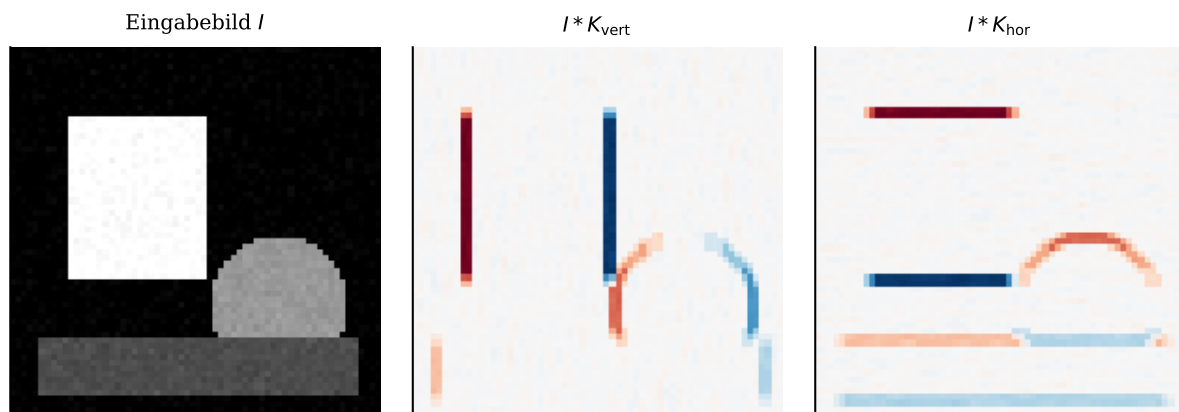


Abbildung 3: Kantendetektion mit festen 3×3 -Filtern. Links das Eingabebild, Mitte die Antwort des vertikalen Kantensfilters, rechts die des horizontalen. Blau/rot: negative/positive Antwort; weiß: keine Änderung.

2.2 Drei Ideen hinter der Faltung

Goodfellow et al. [1] fassen zusammen, was die Faltung gegenüber der vollverbundenen Schicht gewinnt:

Spärliche Interaktionen. Jede Ausgabe hängt nur von M^2 statt von $J \cdot K$ Eingaben ab. Mit $M = 3$ und einem 224×224 -Bild sind das 9 statt 50 176 Verbindungen pro Einheit und damit eine Reduktion um vier Größenordnungen in Speicher und Rechenzeit.

Parameterteilung. Derselbe Kernel wird an jeder Position verwendet. Statt für jede Position eigene Gewichte zu lernen, lernt das Netz einen Filter und wendet ihn überall an. Das ist der Grund, warum die Parameterzahl in Abb. 1 nicht von der Bildgröße abhängt.

Äquivarianz gegenüber Verschiebung. Verschiebt man das Eingabebild um Δ Pixel, verschiebt sich die Feature Map um dieselben Δ Pixel: $(\tau_{\Delta} I) * K = \tau_{\Delta}(I * K)$. Ein Merkmal wird also an jeder Position gleich gut erkannt ohne dass es an jeder Position in den Trainingsdaten vorkommen müsste. Äquivarianz ist *nicht* Invarianz: Die Position bleibt in der Feature Map erhalten. Invarianz („es ist eine Katze, egal wo“) entsteht erst durch Pooling (Abschn. 3).

Merke

Eine Faltungsschicht ist eine vollverbundene Schicht mit zwei harten Nebenbedingungen: Die meisten Gewichte sind *null* (Lokalität), und die übrigen sind *gleich* (Parameterteilung). Goodfellow et al. nennen das einen *unendlich starken Prior*: Das Netz kann diese Annahmen nicht durch Daten widerlegen. Diese Annahme passt gut zu Bildern, Audiodaten und Zeitreihen, weil dort benachbarte Werte tatsächlich zusammenhängen. Bei Tabellendaten trifft das aber nicht zu. Ob eine Größe in Spalte 3 oder Spalte 4 steht, hat keine inhaltliche Bedeutung und es gibt keine 'Nachbarschaft', die eine Faltung sinnvoll ausnutzen könnte. Für solche Daten ist deshalb weiterhin ein MLP die bessere Wahl.

2.3 Padding und Stride

Ohne weitere Maßnahme ist die Feature Map kleiner als das Bild: Ein $M \times M$ -Kernel passt nur an $(J - M + 1) \times (K - M + 1)$ Positionen. Zwei Stellschrauben steuern die Ausgabegröße:

- **Padding P :** Das Bild wird an allen Rändern um P Pixel (typisch mit Nullen, nach Mittelwertsubtraktion) aufgefüllt. Ohne Padding spricht man von *valid*, mit $P = (M - 1)/2$ von *same* Convolution, weil die Feature Map dann die Bildgröße behält. Kernel haben deshalb praktisch immer ungerade Größe (3×3 , 5×5 , 7×7), damit das Padding symmetrisch ist und der Filter ein Zentrumpixel hat.
- **Stride S :** Statt den Filter um ein Pixel weiterzuschieben, springt er um S Pixel. Ein Stride von 2 halbiert die Feature Map in beiden Richtungen (Viertelung der Zahl der Einheiten) und kann so als ein Ersatz für Pooling, den ResNet durchgängig nutzt, eingesetzt werden.

Satz 2.3: Ausgabegröße einer Faltungsschicht

Ein Bild der Größe $J \times K$, gefaltet mit einem $M \times M$ -Kernel, Padding P und Stride S , liefert eine Feature Map der Größe

$$\left\lfloor \frac{J + 2P - M}{S} \right\rfloor + 1 \times \left\lfloor \frac{K + 2P - M}{S} \right\rfloor + 1. \quad (2)$$

Probe: $J = 28$, $M = 5$, $P = 2$, $S = 1$ ergibt 28 (same); $J = 224$, $M = 7$, $P = 3$, $S = 2$ ergibt 112 – die erste Schicht von ResNet.

2.4 Kanäle: Mehrere Filter, mehrere Eingaben

Ein Farbbild hat drei Kanäle (RGB, Rot, Grün, Blau). Der Kernel wird dann zu einem Tensor $M \times M \times C_{\text{in}}$ und summiert über alle Kanäle. Ein einzelner Filter kann nur *ein* Muster detektieren. Deshalb hat jede Faltungsschicht C_{out} unabhängige Filter, deren Feature Maps zu einem Tensor $J' \times K' \times C_{\text{out}}$ gestapelt werden, welcher zu den Eingangskanälen der nächsten Schicht wird. Der Gewichtstensor einer Schicht hat damit die Form $M \times M \times C_{\text{in}} \times C_{\text{out}}$.

Satz 2.4: Parameterzahl einer Faltungsschicht

Eine Faltungsschicht mit C_{out} Filtern der Größe $M \times M$ auf C_{in} Eingangskanälen hat unabhängig von der räumlichen Größe der Eingabe

$$(M^2 C_{\text{in}} + 1) C_{\text{out}} \quad (3)$$

lernbare Parameter (eine Bias pro Ausgangskanal). Die Zahl der Multiplikationen pro Bild ist dagegen $J' K' M^2 C_{\text{in}} C_{\text{out}}$ und wächst mit der Auflösung.

Beispiel 2.5: Parameter und Rechenaufwand in VGG-16

Die erste Schicht von VGG-16 [8] hat 64 Filter 3×3 auf 3 Kanälen: $(9 \cdot 3 + 1) \cdot 64 = 1\,792$ Parameter, aber bei 224×224 Ausgabegröße $224^2 \cdot 27 \cdot 64 \approx 87$ Mio. Multiplikationen. Die erste vollverbundene Schicht am Ende ($7 \cdot 7 \cdot 512 = 25\,088$ Eingaben, 4096 Ausgaben) hat ≈ 103 Mio. Parameter, aber ebenfalls nur ≈ 103 Mio. Multiplikationen. **Faltungsschichten sind rechenintensiv, vollverbundene Schichten sind speicherintensiv**, deshalb werden in modernen Architekturen die großen FC-Schichten durch Global Average Pooling ersetzt (Abschn. 3).

1×1 -Faltungen. Ein Kernel der Größe 1×1 mischt keine räumliche Nachbarschaft, sondern nur die Kanäle: An jeder Position wird der C_{in} -Vektor linear auf C_{out} Dimensionen abgebildet **lin_network_2014**. Er ist die billigste Möglichkeit, die Kanalzahl zu ändern und wird typischerweise genutzt um diese zu reduzieren, bevor eine teure 3×3 -Faltung folgt. GoogLeNet, ResNet-50 und YOLO nutzen ihn systematisch (Abschn. 4).

2.5 Rezeptives Feld und Tiefe

Eine Einheit in der ersten Schicht sieht $M \times M$ Pixel. Eine Einheit in der zweiten Schicht sieht $M \times M$ Einheiten der ersten Schicht und damit einen $(2M - 1) \times (2M - 1)$ -Ausschnitt des Bildes. Mit jeder Schicht wächst das *effektive rezeptive Feld* (Abb. 4), Pooling und Stride beschleunigen dieses Wachstum. Deshalb kann eine Ausabeeinheit tief im Netz auf das ganze Bild reagieren, obwohl jede einzelne Schicht lokal ist. Zwei gestapelte 3×3 -Schichten haben dasselbe rezeptive Feld wie eine 5×5 -Schicht, aber $2 \cdot 9 = 18$ statt 25 Gewichte pro Kanal und eine zusätzliche Nichtlinearität. Diese Beobachtung ist das Designprinzip von VGG: viele kleine Filter statt weniger großer [8].

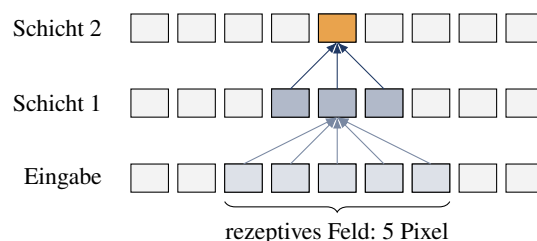


Abbildung 4: Wachstum des rezeptiven Feldes mit der Tiefe (eindimensional, Kernelbreite 3, Stride 1). Die orange Einheit in Schicht 2 hängt von 3 Einheiten in Schicht 1 und damit von 5 Eingabepixeln ab. (nach [4, Abb. 10.9])

2.6 Faltung in PyTorch

Listing 1 zeigt eine Faltungsschicht mit Batch Normalization und ReLU. Wichtig: Bei Batch Normalization in Faltungsschichten werden Mittelwert und Varianz *pro Kanal* über Batch *und* alle räumlichen Positionen berechnet, damit die Statistik nicht von der Position abhängt und die Äquivarianz erhalten bleibt [4], [10].

Listing 1: Eine Faltungsschicht in PyTorch. `padding=1` bei Kernelgröße 3 ergibt eine *same* Convolution; `bias=False`, weil die nachfolgende BatchNorm einen eigenen Offset β lernt.

```
import torch, torch.nn as nn

conv = nn.Sequential(
    nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3,
              stride=1, padding=1, bias=False),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True),
)

x = torch.randn(32, 3, 160, 160)      # (N, C, H, W)
y = conv(x)
```

```
print(y.shape) # torch.Size([32, 64, 160, 160])
print(sum(p.numel() for p in conv.parameters())) # 1728 + 2*64 = 1856
```

Zur Kontrolle: $9 \cdot 3 \cdot 64 = 1728$ Faltungsgewichte plus $2 \cdot 64$ BN-Parameter (γ, β). Die laufenden Mittelwerte der BatchNorm zählen nicht als Parameter (*buffers*), werden aber mit dem Modell gespeichert.

3 Pooling, Invarianz und das vollständige CNN

3.1 Pooling

Für die Klassifikation soll das Netz am Ende *invariant* gegenüber kleinen Verschiebungen sein. Ob also das Auge zwei Pixel weiter links liegt, darf die Antwort „Gesicht“ nicht ändern. Die Faltung liefert nur Äquivarianz. Invarianz gegenüber kleinen Verschiebungen erzeugt *Pooling*: Eine Pooling-Einheit fasst ein kleines Fenster der Feature Map (typisch 2×2 , Stride 2) mit einer festen Funktion ohne lernbare Parameter zusammen.

Definition 3.1: Max-Pooling und Average-Pooling

Für eine Feature Map C und Fenstergröße M_p mit Stride S_p ist

$$\text{MaxPool}(C)(j, k) = \max_{0 \leq l, m < M_p} C(S_p j + l, S_p k + m),$$

$$\text{AvgPool}(C)(j, k) = \frac{1}{M_p^2} \sum_{0 \leq l, m < M_p} C(S_p j + l, S_p k + m).$$

Pooling wird kanalweise angewendet: Aus $64 \times 64 \times 8$ wird mit 2×2 /Stride 2 ein Tensor $32 \times 32 \times 8$.

Max-Pooling merkt sich, *ob* ein Merkmal im Fenster vorkommt und wie stark, aber nicht mehr, *wo genau*, was genau die gewünschte lokale Invarianz erzeugt [1]. Nebenbei halbiert es die räumliche Auflösung und damit die Rechenlast der folgenden Schichten. Das ist der Grund für das typische Profil eines CNN. Die räumliche Größe der Feature Maps sinkt von Schicht zu Schicht, während die Kanalzahl steigt (z. B. $224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$ bei $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ Kanälen in VGG-16, Abb. 6). Die Gesamtzahl der Einheiten pro Schicht bleibt dabei in derselben Größenordnung denn das Netz tauscht räumliche Auflösung gegen semantische Tiefe.

Global Average Pooling. Am Ende eines modernen CNN steht meist kein Flatten + große FC-Schicht mehr, sondern *Global Average Pooling*. Dabei wird jede Feature Map über alle Positionen gemittelt. Aus $7 \times 7 \times 512$ wird ein Vektor der Länge 512, auf den eine einzige lineare Schicht folgt. Das spart die ≈ 103 Mio. Parameter aus Beispiel 2.5, macht das Netz unabhängig von der Eingabegröße und wirkt regularisierend **lin_network_2014**. Dieses Prinzip wird bspw. von ResNet genutzt **he_deep_2016**. Für das Trägermodell bedeutet es, dass der „Kopf“, den wir beim Transfer Learning austauschen, eine einzelne Schicht $512 \rightarrow 5$ mit 2565 Parametern ist.

Pooling oder Stride? Strided Convolution (Abschn. 2.3) verkleinert die Feature Map ebenfalls, aber mit gelernten Gewichten. ResNet und die meisten Architekturen seit 2015 verwenden fast nur noch Stride-2-Faltungen und ein einziges Max-Pooling am Anfang; VGG und LeNet nutzen Pooling nach jedem Block. In der Praxis funktionieren beide Ansätze. Pooling ist parameterfrei und robust, Stride ist dafür flexibler.

3.2 Anatomie eines CNN

Ein vollständiges CNN für Klassifikation besteht aus zwei Teilen (Abb. 5):

1. **Backbone (Feature Extractor).** Eine Folge von Blöcken aus Faltung, Normalisierung, Nichtlinearität und Downsampling. Frühe Schichten lernen Kanten und Farbübergänge, mittlere Texturen und Teile,

späte ganze Objekte (Abschn. 8.3). Der Backbone macht 90–99 % der Rechenzeit aus und ist das, was beim Transfer Learning *übernommen* wird.

2. **Kopf (Head).** Global Average Pooling oder Flatten, gefolgt von einer oder wenigen linearen Schichten und Softmax. Der Kopf ist aufgabenspezifisch und wird beim Transfer Learning *ersetzt*.

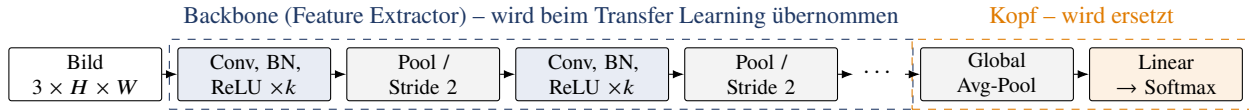


Abbildung 5: Anatomie eines CNN für Klassifikation: Backbone aus Faltungsblocken mit sinkender räumlicher Auflösung und steigender Kanalzahl, gefolgt von einem kleinen aufgabenspezifischen Kopf.

3.3 LeNet-5: Das erste CNN für MNIST

LeNet-5 [6] ist die Blaupause aller späteren CNNs und das Modell des ersten Lab-Teils. Es besteht aus zwei Faltungs-Pooling-Blöcken und drei vollverbundenen Schichten (Tab. 2). Mit rund 60 000 Parametern erreicht es auf MNIST nach einer Epoche etwa 98,8 % Testgenauigkeit. Zum Vergleich lag das MLP aus dem letzten Skript nach einer Epoche bei etwa 96 % [5]. Der Gewinn kommt nicht aus mehr Parametern (das MLP 784–128–128–10 hat mit $\approx 118\,000$ fast doppelt so viele), sondern aus dem passenden Prior.

Schicht	Operation	Ausgabe ($C \times H \times W$)	Kernel/Stride	Parameter
Eingabe	–	$1 \times 28 \times 28$	–	–
C1	Conv 5×5 , Padding 2, ReLU	$6 \times 28 \times 28$	5/1	$(25 \cdot 1 + 1) \cdot 6 = 156$
S2	Max-Pool	$6 \times 14 \times 14$	2/2	0
C3	Conv 5×5 , ReLU	$16 \times 10 \times 10$	5/1	$(25 \cdot 6 + 1) \cdot 16 = 2\,416$
S4	Max-Pool	$16 \times 5 \times 5$	2/2	0
F5	Flatten + Linear, ReLU	120	–	$400 \cdot 120 + 120 = 48\,120$
F6	Linear, ReLU	84	–	$120 \cdot 84 + 84 = 10\,164$
Ausgabe	Linear (Logits)	10	–	$84 \cdot 10 + 10 = 850$
Summe				61 706

Tabelle 2: LeNet-5 in der heute üblichen Variante (ReLU statt tanh, Max- statt Average-Pooling). Die Parameterzahlen folgen aus Gl. (3) bzw. der FC-Formel; die Ausgabegrößen aus Gl. (2).

Listing 2: LeNet-5 in PyTorch. Das Modell ersetzt MLP im Trainingsloop aus dem Skript *Vom Perzeptron zum Deep Network*; Loop, Optimierer und Loss bleiben unverändert – nur das `x.view(-1, 784)` entfällt, weil das Netz das Bild als $1 \times 28 \times 28$ erwartet.

```
class LeNet(nn.Module):
    def __init__(self, n_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(6, 16, kernel_size=5), nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 5 * 5, 120), nn.ReLU(),
            nn.Linear(120, 84), nn.ReLU(),
            nn.Linear(84, n_classes), # Logits -> CrossEntropyLoss
        )
    def forward(self, x): # x: (N, 1, 28, 28)
        return self.classifier(self.features(x))
```

In der Praxis

Dimensionen prüfen, bevor man trainiert. Der häufigste Fehler beim Bau eines CNN ist eine falsche Eingabegröße der ersten linearen Schicht ($16 * 5 * 5$). Rechnen Sie die Kette mit Gl. (2) durch *oder* lassen Sie sich die Form geben: `print(model.features(torch.zeros(1, 1, 28, 28)).shape)`. Ein zweiter Klassiker: `nn.Conv2d` erwartet (N, C, H, W) ; ein mit PIL oder OpenCV geladenes Bild liegt als (H, W, C) vor. `torchvision.transforms.ToTensor()` übernimmt die Umordnung und skaliert auf $[0, 1]$.

3.4 Datenaugmentierung für Bilder

Im Skript *Vom Perzeptron zum Deep Network* wurde Augmentierung als Regularisierung eingeführt. Für Bilder ist sie besonders natürlich, weil wir wissen, welche Transformationen das Label *nicht* ändern: horizontale Spiegelung (nicht bei Ziffern denn z.B. eine gespiegelte 3 ist keine 3), zufälliger Ausschnitt und Skalierung, kleine Rotationen, Helligkeits- und Farbjitter. Murphy [5] beschreibt Augmentierung als Ersatz des empirischen Verlusts durch einen Erwartungswert über die Transformationsverteilung, also als Glättung der Trainingsverteilung in Richtungen, die wir für irrelevant halten. AlexNet, VGG und ResNet wurden alle mit zufälligen Ausschnitten und Spiegelungen trainiert, ResNet zusätzlich noch mit Skalenaugmentierung (kürzere Bildseite zufällig in $[256, 480]$ gekürzt) **he_deep_2016**. Für ihr Trägermodell ist Augmentierung Pflicht. Bei 500 Bildern pro Klasse ist das Netz sonst nach wenigen Epochen überangepasst. Zur Testzeit wird *nicht* augmentiert, sondern nur die Größenanpassung und Normalisierung (dieselbe wie im Pretraining!) bleiben.

4 Architekturen: Von LeNet zu ResNet

Zwischen LeNet-5 (1998) und ResNet (2015) liegen 17 Jahre, in denen sich die Bausteine kaum, die Tiefe aber um den Faktor 30 verändert hat. Tabelle 3 ordnet die wichtigsten Architekturen ein und die folgenden Abschnitte erklären, welche Idee jeweils den nächsten Schritt möglich gemacht hat.

Architektur	Jahr	Tiefe	Parameter	Top-5-Fehler	Neue Idee
LeNet-5 [6]	1998	5	60 k	– (MNIST)	Conv + Pooling + FC, Backprop end-to-end
AlexNet [7]	2012	8	60 M	15,3 %	ReLU, Dropout, GPU-Training, Augmentierung
VGG-16 [8]	2014	16	138 M	7,3 %	nur 3×3 -Filter, gleichförmige Blöcke
GoogLeNet szegedy_going_2015	2014	22	6,8 M	6,7 %	Inception-Module, 1×1 -Reduktion, Global Avg-Pool
ResNet-152 he_deep_2016	2015	152	60 M	3,6 % (Ens.)	Residualverbindungen, Batch Norm überall
EfficientNet-B0	2019	–	5,3 M	$\approx 6,7$ %	Compound Scaling von Tiefe, Breite, Auflösung
ConvNeXt-T	2022	–	28 M	$\approx 3,9$ %	CNN mit Transformer-Design-Entscheidungen
ViT-B/16	2020	12	86 M	$\approx 4,7$ %	kein CNN: Bild als Sequenz von Patches

Tabelle 3: Meilensteine der Bildklassifikation. Top-5-Fehler auf ImageNet (Testset des jeweiligen Wettbewerbs, teils Ensembles; die drei unteren Werte sind Einzelmodelle aus den jeweiligen Originalarbeiten, ungefähr). Parameter gerundet. Die drei unteren Zeilen sind die Modelle des erweiterten Labs.

4.1 AlexNet und VGG: Tiefe durch Training ermöglichen

AlexNet [7] war strukturell ein großes LeNet. Fünf Faltungsschichten mit 11×11 -, 5×5 - und 3×3 -Filtern, drei FC-Schichten. Der Beitrag lag im Training. Es wurde ReLU statt Sigmoid eingesetzt (*sechsmal*

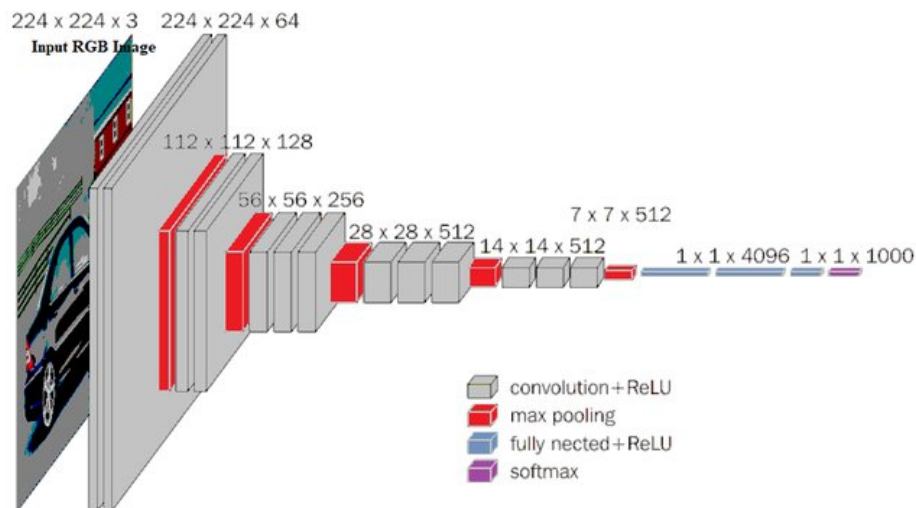


Abbildung 6: VGG-16 [11, Abb. 3]: fünf Faltungsblöcke aus 3×3 -Faltungen (Kanalzahl und räumliche Größe angegeben), jeweils gefolgt von 2×2 -Max-Pooling, dann drei vollverbundene Schichten. 13 Faltungs- und 3 FC-Schichten ergeben die 16 im Namen.

schnellere Konvergenz), Dropout in den FC-Schichten implementiert, Augmentierung genutzt und ein Training auf zwei GPUs über eine Woche durchgeführt. Alles, was im Skript *Vom Perzeptron zum Deep Network* über Optimierung und Regularisierung behandelt wurde, wurde hier erstmals in großem Maßstab kombiniert.

VGG [8] radikalisierte die Architektur. Ausschließlich 3×3 -Faltungen mit *same* Padding, Verdopplung der Kanäle nach jedem 2×2 -Pooling, 16 bzw. 19 gewichtete Schichten (Abb. 6). Der Preis: 138 Mio. Parameter, davon 103 Mio. in der ersten FC-Schicht, und 15,5 GFLOPs pro Bild. Dies resultierte in einer der bis heute teuersten Architekturen. VGG-16 ist trotzdem didaktisch wertvoll, weil man an ihm jede Dimension nachrechnen kann, und es bleibt der Standardvergleichspunkt in der ResNet-Arbeit.

4.2 GoogLeNet: Rechenaufwand durch 1×1 -Faltungen zählen

GoogLeNet *szegedy_going_2015* erreichte mit 22 Schichten und nur 6,8 Mio. Parametern (5 % von VGG) einen besseren Fehler. Zwei Ideen führten zu diesem Ergebnis. Erstens *Inception-Module*, die parallel 1×1 -, 3×3 - und 5×5 -Filter sowie Pooling anwenden und die Ergebnisse konkatenerieren und das Netz wählt selbst die Filtergröße. Zweitens 1×1 -Faltungen *vor* den teuren Filtern, um die Kanalzahl zu reduzieren (Abschn. 2.4), und Global Average Pooling statt FC-Schichten am Ende. YOLO übernimmt genau dieses Rezept mit 1×1 -Reduktionsschichten vor jeder 3×3 -Faltung (Abschn. 6).

4.3 ResNet: Das Degradationsproblem

Nach VGG lag die Vermutung nahe, dass mehr Tiefe automatisch bessere Netze liefert. He et al. *he_deep_2016* zeigten, dass das nicht stimmt. Abbildung 7 (links) zeigt schematisch ihr Experiment. Ein 34-schichtiges *plain*-Netz (VGG-artig, mit Batch Norm) hat einen *höheren Trainingsfehler* als ein 18-schichtiges. Das ist kein Overfitting (Trainingsfehler ist schlechter, nicht der Testfehler). Auch verschwindenden Gradienten scheiden als Erklärung aus. Batch Norm hält die Gradientennormen während des Trainings nachweislich gesund.

Das Argument ist einfach nachzuvollziehen: Nimmt man das 18-Schicht-Netz und hängt 16 Schichten an, die lediglich ihre Eingabe unverändert weitergeben (*Identitätsabbildung*), hat das 34-Schicht-Netz exakt denselben Trainingsfehler. Eine Lösung mit *mindestens* so gutem Fehler existiert also. Der Optimierer findet sie nur nicht: Eine Identitätsabbildung durch einen Stapel nichtlinearer Schichten zu *lernen* ist für SGD überraschend schwer. Das *Degradationsproblem* ist ein Optimierungsproblem, kein Repräsentationsproblem.

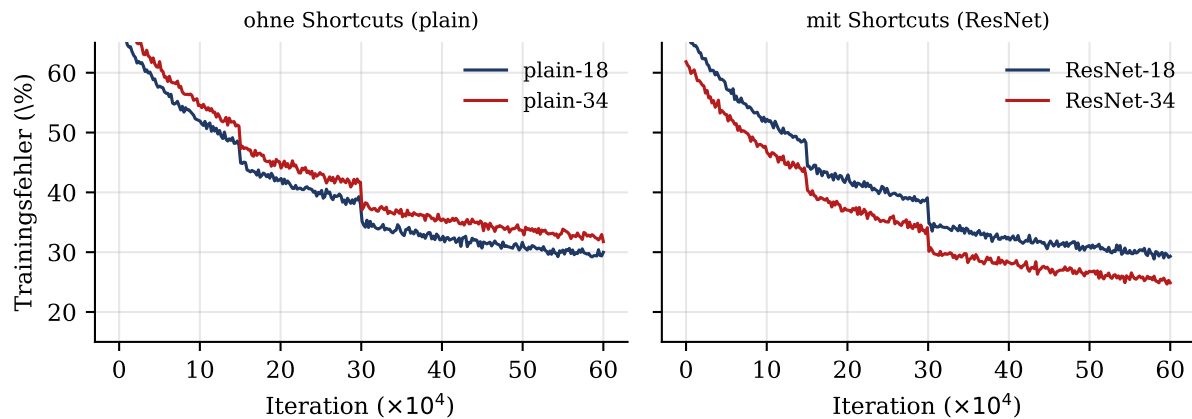


Abbildung 7: Das Degradationsproblem (schematisch nach [2, Abb. 4]). Links: Bei plain-Netzen hat das tiefere Netz den höheren Trainingsfehler. Rechts: Mit Residualverbindungen kehrt sich das Bild um; das 34-Schicht-ResNet ist um 2,8 Prozentpunkte besser als das 18-Schicht-ResNet (Top-1, 10-crop). Die Stufen sind Lernratenabfälle.

4.4 Residual Learning

Die Lösung besteht darin, das Netz nicht die gewünschte Abbildung, sondern ihre *Abweichung von der Identität* lernen zu lassen.

Definition 4.1: Residualblock

Sei $\mathcal{H}(x)$ die Abbildung, die ein Stapel von wenigen Schichten lernen soll. Statt $\mathcal{H}$ direkt zu parametrisieren, lernen die Schichten die *Residualfunktion* $\mathcal{F}(x) := \mathcal{H}(x) - x$, und die Ausgabe des Blocks ist

$$y = \mathcal{F}(x; \{W_i\}) + x, \quad \text{z. B. } \mathcal{F} = W_2 \sigma(W_1 x). \quad (4)$$

Die Addition von x ist die *Shortcut-* oder *Skip-Verbindung*; sie ist parameterfrei und wird elementweise (Kanal für Kanal) ausgeführt. Nach der Addition folgt eine weitere Nichtlinearität. Stimmen die Dimensionen von x und $\mathcal{F}(x)$ nicht überein (bei Verdopplung der Kanäle mit Stride 2), wird x durch eine 1×1 -Faltung W_s projiziert: $y = \mathcal{F}(x) + W_s x$.

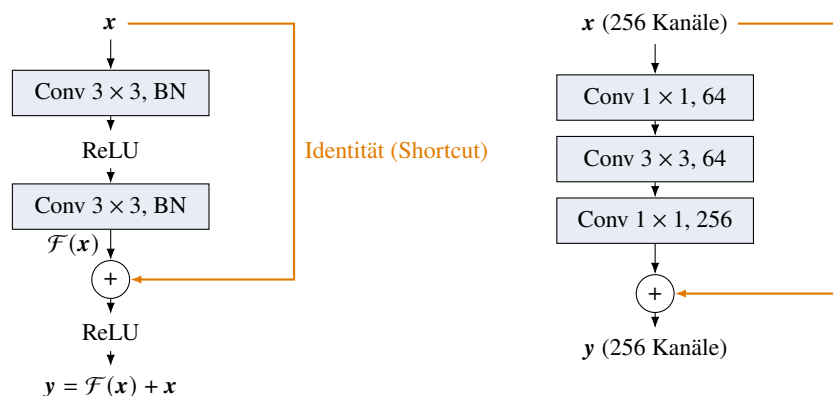


Abbildung 8: Links: Basis-Residualblock von ResNet-18/34 (zwei 3×3 -Faltungen, je mit Batch Norm; ReLU zwischen den Faltungen und nach der Addition). Rechts: Bottleneck-Block von ResNet-50/101/152 – 1×1 reduziert die Kanäle, 3×3 arbeitet auf der reduzierten Breite, 1×1 stellt sie wieder her. Beide Blöcke haben ähnliche Rechenzeit; der Bottleneck erlaubt viermal breitere Ein- und Ausgaben. (Nach [2, Abb. 2 und 5])

Der Erfolg von Residualverbindungen lässt sich aus zwei Perspektiven begründen. Zum einen rein optimierungstheoretisch: Wenn die optimale Abbildung nahe der Identität liegt, muss das Netz nur noch eine kleine Störung $\mathcal{F} \approx \mathbf{0}$ lernen, und das ist für SGD einfach: Es genügt, die Gewichte gegen null zu treiben. He et al. zeigen empirisch, dass die gelernten Residualfunktionen tatsächlich kleine Antworten haben und dass diese Antworten mit wachsender Tiefe noch kleiner werden denn jede Schicht eines tiefen ResNet verändert das Signal nur wenig **he_deep_2016**. Eine weitere Perspektive ist die der Gradientenausbreitung: Die Ableitung von Gl. (4) nach $\mathbf{x}$ ist $\partial y / \partial \mathbf{x} = \mathbf{I} + \partial \mathcal{F} / \partial \mathbf{x}$. Die Jacobi-Kette der Rückwärtsrechnung wird zu einem Produkt von Faktoren der Form $(\mathbf{I} + \text{klein})$ statt (klein) und der Gradient hat immer einen direkten Pfad zur Eingabe. Residualverbindungen wurden bereits als Gegenmaßnahme gegen verschwindende Gradienten genannt. ResNet kann als der Beleg im großen Maßstab gesehen werden. Dieselbe Idee wird in *Sequenzmodellen & Transformern* weiter verfolgt. Jeder Transformer-Block ist im Kern ein Residualblock.

4.5 Die ResNet-Familie

Tabelle 4 zeigt die fünf Standardvarianten. Alle beginnen mit einer 7×7 -Faltung (Stride 2, 64 Kanäle) und einem Max-Pooling, dann folgen vier Stufen (conv2_x bis conv5_x) mit Feature-Map-Größen $56 \rightarrow 28 \rightarrow 14 \rightarrow 7$ und Kanalzahlen $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ (bzw. das Vierfache am Bottleneck-Ausgang). Downsampling geschieht durch Stride 2 in der jeweils ersten Faltung einer Stufe. Am Ende steht jeweils ein Global Average Pooling und eine lineare Schicht. Es gibt *keine* versteckten FC-Schichten und kein Dropout **he_deep_2016**.

	ResNet-18	ResNet-34	ResNet-50	ResNet-101	ResNet-152
Blocktyp	Basis	Basis	Bottleneck	Bottleneck	Bottleneck
Blöcke je Stufe	2, 2, 2, 2	3, 4, 6, 3	3, 4, 6, 3	3, 4, 23, 3	3, 8, 36, 3
Parameter	11,7 M	21,8 M	25,6 M	44,5 M	60,2 M
GFLOPs (224 px)	1,8	3,6	3,8	7,6	11,3
Top-1 / Top-5 (10-crop, Val.)	27,9 / –	24,5 / 7,5	22,9 / 6,7	21,8 / 6,1	21,4 / 5,7
Gewichtsdatei (fp32)	45 MB	84 MB	98 MB	171 MB	231 MB

Tabelle 4: Die ResNet-Familie für ImageNet nach [2, Tab. 1 und 3] (Fehler in %; ResNet-34 mit Option B, d. h. Projektions-Shortcuts nur bei Dimensionswechsel). Zum Vergleich: VGG-16 hat 15,3 GFLOPs und 9,3 % Top-5-Fehler unter gleichem Protokoll. ResNet-152 ist achtmal tiefer als VGG-19 und trotzdem billiger.

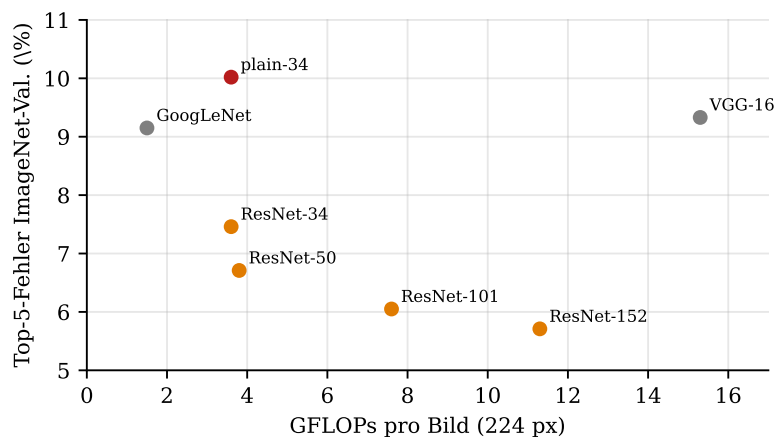


Abbildung 9: Genauigkeit gegen Rechenaufwand auf ImageNet (Zahlen aus [2]). Das plain-34-Netz kostet dasselbe wie ResNet-34, ist aber schlechter als VGG-16; mit Shortcuts liegt jede Vergrößerung von ResNet auf einer sinkenden Kurve. Für das Trägermodell wählen wir bewusst den kleinsten Punkt links unten, ResNet-18 (nicht abgebildet, 1,8 GFLOPs).

4.6 Seit ResNet

Die Grundidee des ResNet wurde immer weiter verfeinert. *DenseNet* verbindet jede Schicht mit allen folgenden, *MobileNet* und *EfficientNet* ersetzen die 3×3 -Faltung durch *depthwise separable* Faltungen (erst räumlich pro Kanal, dann 1×1 über Kanäle) und senken die Rechenlast um den Faktor 8–9 bei ähnlicher Genauigkeit (die Wahl für mobile und Edge-Geräte). *ConvNeXt* überträgt Design-Entscheidungen des Vision Transformers (große 7×7 -Kernel, Layer Norm, GELU) zurück auf CNNs und ist auf ImageNet mit ViTs gleichauf. Der *Vision Transformer* selbst zerlegt das Bild in 16×16 -Patches und verarbeitet sie als Sequenz mit Self-Attention. Dabei wird auf den Faltungs-Prior verzichtet weshalb sehr große Vortrainingsdaten benötigt werden, um konkurrenzfähig zu sein.

4.7 Eine Architektur auswählen

Die Genauigkeit auf ImageNet ist nur ein Kriterium bei der begründeten Auswahl einer Architektur für bestimmte Anwendungsfälle. In der Praxis dominiert fast immer eines der folgenden fünf Kriterien (Tab. 5)

Kriterium	Leitfrage	Konsequenz
Datenmenge	Wie viele gelabelte Bilder pro Klasse?	$< 1\,000$ /Klasse: Transfer Learning mit eingefrorenem Backbone; $> 10^5$ gesamt: Fine-Tuning oder Training von Null denkbar
Zielhardware, Latenz	Wo läuft die Inferenz, mit welchem Budget pro Bild?	CPU/Edge, < 50 ms: ResNet-18, MobileNet, EfficientNet-B0; Server-GPU: ResNet-50, ConvNeXt, ViT
Auflösung, Objektgröße	Reichen 160–224 px, oder sind Details entscheidend (Defekte, Zellen)?	höhere Auflösung $\Rightarrow$ quadratisch mehr FLOPs; ggf. Kacheln statt Verkleinern
Aufgabe	Klassifikation, Detektion, Segmentierung?	Backbone bleibt; Kopf und Loss ändern sich (Abschn. 6 and 7)
Domänenabstand	Wie ähnlich sind die Bilder zu ImageNet?	Fotos: ImageNet-Gewichte übertragen sehr gut; Röntgen, Satellit, Mikroskopie: mehr Schichten feinjustieren, ggf. Domänen-Pretraining
Lifecycle-Kosten	Muss das Modell versioniert, deployt, überwacht, erklärt werden?	jede Verdopplung der Parameter verdoppelt Speicher, Docker-Image, Latenz und XAI-Rechenzeit

Tabelle 5: Kriterien für die Architekturauswahl. Die letzte Zeile ist der Grund, warum das Trägermodell ein ResNet-18 ist.

Merke

Begründung für das Trägermodell. ResNet-18: 11,7 Mio. Parameter, 44 MB Gewichte, ≈ 30 ms Inferenz pro Bild auf einer Laptop-CPU. Damit bleiben der Docker-Container (≈ 1 GB Image), die Drift-Simulation über hunderte Bilder und die SHAP-Erklärungen (die das Modell hunderte Male pro Bild auswerten) auf der Referenzhardware machbar. ResNet-50 wäre auf Imagenette zwei Prozentpunkte genauer, würde aber jede dieser Ketten um den Faktor 2–3 verteuern. Genauigkeit war hier nicht das bindende Kriterium.

5 Transfer Learning: Das Trägermodell entsteht

5.1 Warum Vortraining übertragbar ist

Ein auf ImageNet trainiertes Netz hat 1,28 Mio. Bilder gesehen und dabei gelernt, was Kanten, Texturen, Fell, Räder, Himmel und Gebäudefassaden sind. Diese Merkmale sind nicht ImageNet-spezifisch, sondern Eigenschaften natürlicher Bilder. Frühe Schichten lernen fast unabhängig vom Datensatz dieselben Gabor-artigen Filter (Abschn. 8.3). Erst späte Schichten werden aufgabenspezifisch **yosinski_how_2014**. Hat man für die eigene Aufgabe nur wenige hundert Bilder pro Klasse, wäre es Verschwendung, diese Merkmale erneut aus Daten zu lernen, zumal das mit so wenigen Daten gar nicht gelänge (Abb. 10). Stattdessen übernimmt man den vortrainierten Backbone und trainiert nur, was aufgabenspezifisch ist. Murphy [5] formuliert das als zwei Phasen: *Pretraining* auf einem großen Quelldatensatz $\mathcal{D}_p$ (gelabelt oder ungelabelt), dann *Fine-Tuning* auf dem kleinen Zieldatensatz $\mathcal{D}_q$.

Definition 5.1: Transfer Learning durch Modellchirurgie

Sei $p(y | \mathbf{x}) = \text{softmax}(\mathbf{W}_2 h(\mathbf{x}; \theta_b) + \mathbf{b}_2)$ das vortrainierte Modell mit Feature Extractor h (Backbone-Parameter θ_b) und Kopf $(\mathbf{W}_2, \mathbf{b}_2)$ für die Quellklassen $\mathcal{Y}_p$ (z. B. 1 000 ImageNet-Klassen). Für die Zielaufgabe mit Klassen $\mathcal{Y}_q$ wird der Kopf entfernt und durch einen neuen, zufällig initialisierten Kopf $(\mathbf{W}_3, \mathbf{b}_3)$ mit $|\mathcal{Y}_q|$ Ausgaben ersetzt:

$$q(y | \mathbf{x}) = \text{softmax}(\mathbf{W}_3 h(\mathbf{x}; \theta_b) + \mathbf{b}_3). \quad (5)$$

Anschließend gibt es zwei Regime [5]:

- **Feature-Extraktion (linear probe):** θ_b bleibt eingefroren; nur $(\mathbf{W}_3, \mathbf{b}_3)$ wird trainiert. Das Modell ist dann *linear in seinen freien Parametern*. Es besteht ein konvexes Problem, das in Sekunden konvergiert und mit sehr wenigen Daten auskommt.
- **Fine-Tuning:** θ_b (oder ein Teil davon, typischerweise die letzten Stufen) wird mitgelernt, mit einer *deutlich kleineren Lernrate* als der Kopf, damit sich die vortrainierten Merkmale nicht zu weit von ihrem Ausgangspunkt entfernen.

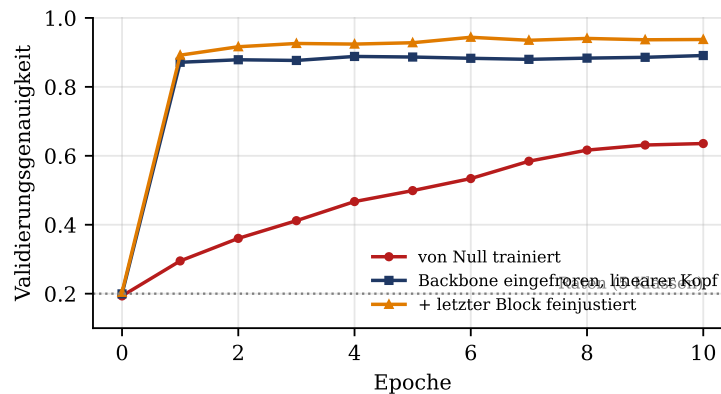


Abbildung 10: Typische Validierungskurven auf einem kleinen Datensatz (schematisch, 5 Klassen, ≈ 500 Bilder/Klasse). Von Null trainiert bleibt das Netz weit hinter dem vortrainierten zurück; der lineare Kopf auf eingefrorenem Backbone ist schon nach einer Epoche gut; das Freischalten des letzten Blocks bringt weitere Prozentpunkte.

5.2 Welche Schichten trainieren?

Die Entscheidung folgt zwei Größen: der Menge der Zieldaten und dem Abstand zwischen Quell- und Zieldomäne (Tab. 6). Yosinski et al. **yosinski_how_2014** zeigen, dass die Übertragbarkeit gelernter Merkmale von Quell- auf Zieldomäne mit der Schichttiefe abnimmt und mit dem Domänenabstand sinkt. Das Fine-Tuning der übertragenen Schichten gleicht die Verluste aus, sobald genug Daten vorhanden sind.

	Zieldomäne ähnlich (Fotos, Alltag)	Zieldomäne fern (Röntgen, Satellit, Mikroskopie)
wenig Daten ($< 10^3$ /Klasse)	Backbone einfrieren, linearen Kopf trainieren; ggf. letzte Stufe freischalten	Backbone einfrieren, aber Features aus <i>mittleren</i> Schichten nutzen; starke Augmentierung; Ergebnis kritisch prüfen
viel Daten ($> 10^4$ /Klasse)	gesamtes Netz feinjjustieren, Backbone-Lernrate $\epsilon/10$	gesamtes Netz feinjjustieren; Alternative: Domänen-Pretraining (selbstüberwacht) auf ungelabelten Zielbildern

Tabelle 6: Faustregeln für das Transfer-Regime nach Datenmenge und Domänenabstand.

Zwei technische Details entscheiden in der Praxis über Erfolg oder Misserfolg:

1. **Vorverarbeitung wie im Pretraining.** Die torchvision-Gewichte erwarten RGB-Bilder, skaliert auf $[0, 1]$ und normalisiert mit ImageNet-Mittelwert (0,485, 0,456, 0,406) und -Standardabweichung (0,229, 0,224, 0,225) je Kanal. Eine andere Normalisierung verschiebt die Eingabevertelung aller Schichten und kostet Prozentpunkte (Beispiel für *Data Drift*).
2. **Batch Norm im eingefrorenen Backbone.** Die laufenden Statistiken der BN-Schichten stammen aus ImageNet. Beim Fine-Tuning mit kleinen Batches (16–32) sind neu geschätzte Batchstatistiken verrauscht und man belässt üblicherweise die BN-Schichten des eingefrorenen Teils im `eval()`-Modus. He et al. verfahren für die Detektion genauso: BN-Statistiken werden nach dem Pretraining fixiert und die Schichten damit zu affinen Abbildungen [2].

5.3 Rezept und Implementierung

Algorithm 1 fasst das Vorgehen zusammen, das im Lab auf ResNet-18 und Imagenette angewendet wird, listing 3 zeigt den PyTorch-Kern dazu.

Voraussetzung: vortrainierter Backbone $h(\cdot; \theta_b)$, Zieldaten $\mathcal{D}_q$ mit C_q Klassen

Schritt 1 – Kopf ersetzen: letzte lineare Schicht durch $\text{Linear}(d_h, C_q)$ ersetzen, zufällig initialisiert

Schritt 2 – Feature-Extraktion: θ_b einfrieren (`requires_grad=False`); Features $h(x_n)$ für alle n *einmal* berechnen und cachen; Kopf mit Adam, $\epsilon = 10^{-3}$, 10–20 Epochen auf den gecachten Features trainieren (Sekunden)

Schritt 3 – Validieren: Genauigkeit und Konfusionsmatrix auf $\mathcal{D}_{\text{val}}$; ist das Ziel erreicht, fertig

Schritt 4 – Partielles Fine-Tuning: letzte Stufe des Backbone (`layer4`) freischalten; zwei Parametergruppen: Kopf $\epsilon = 10^{-3}$, Backbone $\epsilon = 10^{-4}$; mit Augmentierung 3–5 Epochen trainieren; BN-Schichten im `eval()`-Modus

Schritt 5 – Early Stopping auf dem Validierungsverlust; bestes Modell mit Konfiguration, Seed, Klassenliste und Normalisierungsparametern speichern

Algorithm 1: Transfer Learning in zwei Stufen (Feature-Extraktion, dann partielles Fine-Tuning).

Listing 3: Modellchirurgie und Parametergruppen für ResNet-18 in torchvision. `model.fc` ist der Kopf (512 $\rightarrow$ 1000); er wird durch 512 $\rightarrow$ 5 ersetzt.

```
import torch, torch.nn as nn
from torchvision.models import resnet18

model = resnet18(weights=None)
model.load_state_dict(torch.load("./resnet18-f37072fd.pth"))

# Schritt 1: Kopf ersetzen (512 -> 5 Klassen)
model.fc = nn.Linear(model.fc.in_features, 5)
```

```
# Schritt 2: Backbone einfrieren, nur Kopf trainieren
for name, p in model.named_parameters():
    p.requires_grad = name.startswith("fc.")
opt = torch.optim.Adam(model.fc.parameters(), lr=1e-3)

# Schritt 4: layer4 freischalten, zwei Lernraten
for p in model.layer4.parameters():
    p.requires_grad = True
opt = torch.optim.Adam([
    {"params": model.fc.parameters(), "lr": 1e-3},
    {"params": model.layer4.parameters(), "lr": 1e-4},
])

def train_mode(model):
    model.train()
    for m in model.modules():
        # BN-Statistiken einfrieren
        if isinstance(m, nn.BatchNorm2d):
            m.eval()
```

In der Praxis

Features cachen. Bei eingefrorenem Backbone hängt $h(\mathbf{x}_n)$ nicht von den trainierbaren Parametern ab. Berechnen Sie die 512-dimensionalen Features einmal für alle Bilder (ResNet-18, 160 px: ≈ 3 Min. auf 4 Kernen für 3 000 Bilder), speichern Sie sie als Tensor, und trainieren Sie den Kopf darauf. Jede Epoche kostet dann Millisekunden statt Minuten, und Sie können Lernrate, Weight Decay und Epochenzahl in Sekunden rastern. Der Preis: Bei gecachten Features gibt es keine Augmentierung (jede Augmentierung erzeugt neue Features). Für Schritt 4 muss daher wieder über Bilder trainiert werden.

5.4 Das Trägermodell: Festlegung

Ab hier ist das Modell nicht mehr nur ein Lab-Ergebnis, sondern ein Artefakt, welches durch das Semester wandert. Damit das gelingt, ist seine Schnittstelle jetzt festzulegen:

- **Architektur:** ResNet-18, ImageNet-Gewichte (torchvision IMAGENET1K_V1), Kopf $512 \rightarrow 5$; Backbone bis layer3 eingefroren, layer4 feinjustiert.
- **Daten:** Imagenette-Teilmenge, 5 Klassen, 160 px, $\approx 2\,500$ Trainings- und 500 Validierungsbilder, feste Aufteilung per Dateiliste (kein zufälliger Split zur Laufzeit).
- **Eingabe:** RGB, kürzere Seite 160 px, Center-Crop 160×160 , ImageNet-Normalisierung. Diese Vorverarbeitung ist Teil des Modells und wird mit ihm versioniert.
- **Ausgabe:** Logits der Länge 5 in fester Klassenreihenfolge; Softmax erst im Service.
- **Ablage:** models/traeger_v1.pt (State Dict), daneben traeger_v1.yaml mit Klassenliste, Normalisierung, Seed, Hyperparametern und Validierungsmetriken.
- **Zielgenauigkeit:** $\geq 90\%$ auf der Validierungsmenge.

Im Skript *Daten-Engineering für ML* werden diese Daten mit DVC versioniert, im Skript *MLOps: Vom Notebook zum Produktivsystem* wird das Modell hinter einen /predict-Endpunkt gestellt, im Skript *Monitoring & Drift* wird seine Eingabeverteilung überwacht und im Skript *Erklärbare KI & Compliance* werden seine Entscheidungen mit Grad-CAM und SHAP erklärt und in einer Model Card dokumentiert. Vermeiden Sie deshalb jede Unsauberkeit wie bspw. ein Split ohne Seed, eine Normalisierung im Notebook statt im Code, eine Klassenreihenfolge, die von `os.listdir` abhängt.

6 Objektdetektion

6.1 Bounding Boxes und Intersection over Union (IoU)

Bei der Detektion soll das Netz nicht nur sagen, *was* im Bild ist, sondern für jedes Objekt eine *Bounding Box* $\mathbf{b} = (b_x, b_y, b_w, b_h)$ (Mittelpunkt, Breite, Höhe) und eine Klasse ausgeben. Die Anzahl der Objekte ist unbekannt und variiert. Die Ausgabe hat dementsprechend keine feste Länge. Genau das macht Detektion architektonisch schwieriger als Klassifikation.

Zunächst braucht man ein Maß dafür, ob eine vorhergesagte Box „richtig“ ist. Die Fläche der Überlapung allein taugt nicht: Sie hängt von der Objektgröße ab und bestraft nicht, wenn die Vorhersage viel zu groß ist [4].

Definition 6.1: Intersection over Union (IoU)

Für zwei Boxen A und B (als Pixelmengen) ist

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1]. \quad (6)$$

Eine Vorhersage gilt üblicherweise als Treffer, wenn ihre IoU mit einer Ground-Truth-Box derselben Klasse $\geq 0,5$ ist [12]. Die Standardmetrik *mean Average Precision* (mAP) mittelt über alle Klassen die Fläche unter der Precision-Recall-Kurve, die man erhält, wenn man den Konfidenz-Schwellwert variiert; COCO mittelt zusätzlich über IoU-Schwellen von 0,5 bis 0,95 [13].

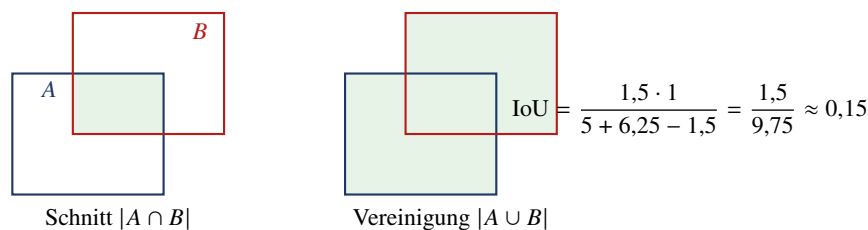


Abbildung 11: Intersection over Union: Verhältnis der Schnittfläche (links) zur Vereinigungsfläche (rechts). Im Beispiel überlappen sich die Boxen zwar, die IoU ist mit 0,15 aber weit unter der üblichen Trefferschwelle von 0,5.

6.2 Der klassische Weg: Klassifikator plus Suche

Bis 2015 hatten alle Detektoren dieselbe Grundform: einen *Klassifikator*, der für einen Bildausschnitt „Objekt der Klasse c “ oder „Hintergrund“ sagt, und eine *Suchstrategie*, die Ausschnitte vorschlägt [3].

- **Sliding Window.** Ein Fenster fester Größe wird in Schritten über das Bild (und über mehrere Skalierungen des Bildes) geschoben und jeder Ausschnitt klassifiziert. Da die Faltung selbst ein gleitendes Fenster ist, lässt sich das effizient als *vollständig konvolutionale* Auswertung implementieren: Ein Netz, das auf 224×224 trainiert wurde, kann auf ein größeres Bild angewendet werden und liefert dann ein Gitter von Vorhersagen statt einer einzelnen [4], [14].
- **Region Proposals (R-CNN-Familie).** Statt alle Positionen zu prüfen, schlägt ein Vorverfahren (Selective Search, $\approx 2\,000$ Regionen) oder ein eigenes kleines Netz (Region Proposal Network in Faster R-CNN) wenige aussichtsreiche Regionen vor, die der Klassifikator dann bewertet und deren Boxen er per Regression verfeinert [15], [16], [17].

Beide Wege erzeugen mehrere Detektionen desselben Objekts an leicht verschobenen Positionen. *Non-Max Suppression* (NMS) räumt auf: Pro Klasse wird die Box mit der höchsten Konfidenz akzeptiert, alle Boxen mit $\text{IoU} > 0,5$ zu ihr werden verworfen, dann wird mit der nächstbesten verbliebenen Box fortgefahren, bis keine mehr übrig ist [4].

Der Nachteil dieser Pipelines ist ihre Zerlegung in Einzelteile: Proposal-Generator, Feature-Extraktor, Klassifikator, Box-Regressor und NMS werden getrennt entworfen und teils getrennt trainiert. R-CNN

brauchte über 40 Sekunden pro Bild; Fast R-CNN mit 0,5 Bildern pro Sekunde war immer noch weit von Echtzeit entfernt [3].

6.3 YOLO: Detektion als Regression

Redmon et al. [3] formulierten die Detektion als *ein einziges Regressionsproblem*: von den Pixeln direkt zu Boxkoordinaten und Klassenwahrscheinlichkeiten, in einem Durchlauf eines einzigen Netzes: *You Only Look Once (YOLO)*.

Definition 6.2: YOLO-Ausgabestruktur

Das Bild wird in ein $S \times S$ -Gitter geteilt. Die Gitterzelle, in die der *Mittelpunkt* eines Objekts fällt, ist für dieses Objekt zuständig. Jede Zelle sagt vorher:

- B Boxen, jede mit fünf Zahlen $(x, y, w, h, \text{Konfidenz})$: Mittelpunkt relativ zur Zelle, Breite und Höhe relativ zum Bild, und eine Konfidenz, die $\text{Pr}(\text{Objekt}) \cdot \text{IoU}_{\text{pred}}^{\text{truth}}$ schätzen soll. Hier steht null, wenn kein Objekt in der Zelle liegt, sonst die IoU der Box mit der Ground Truth
- C bedingte Klassenwahrscheinlichkeiten $\text{Pr}(\text{Klasse}_i \mid \text{Objekt})$, immer *ein* Satz pro Zelle, unabhängig von B .

Die Ausgabe ist ein Tensor der Größe $S \times S \times (5B + C)$. Zur Testzeit ergibt das Produkt $\text{Pr}(\text{Klasse}_i \mid \text{Objekt}) \cdot \text{Pr}(\text{Objekt}) \cdot \text{IoU}$ einen klassenspezifischen Konfidenzwert je Box, der sowohl die Klassenwahrscheinlichkeit als auch die Boxqualität codiert. Für PASCAL VOC [18]: $S = 7$, $B = 2$, $C = 20$, also $7 \times 7 \times 30$.

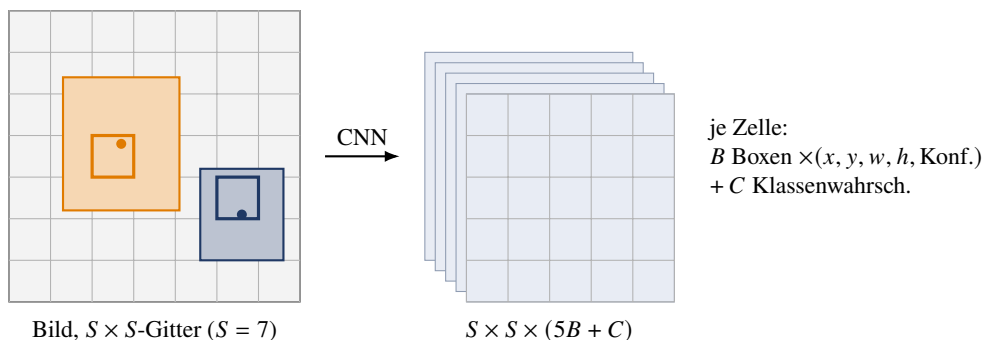


Abbildung 12: YOLO: Das Bild wird in $S \times S$ Zellen geteilt; die Zelle, die den Mittelpunkt eines Objekts enthält (dick umrandet), ist für dessen Box und Klasse zuständig. Die Ausgabe ist ein Tensor fester Größe wobei die variable Zahl der Objekte über das Gitter und Konfidenzschwellen abgebildet wird. (nach [3, Abb. 2])

Netz. Der Backbone ist von GoogLeNet inspiriert: 24 Faltungsschichten mit 1×1 -Reduktion vor 3×3 -Faltungen, vortrainiert auf ImageNet bei 224×224 , für die Detektion auf 448×448 verdoppelt (Detektion braucht Details). Zwei FC-Schichten erzeugen den $7 \times 7 \times 30$ -Tensor. Leaky ReLU ($\varphi(x) = x$ für $x > 0$, sonst $0,1x$) in allen Schichten außer der letzten, die linear ist [3].

Verlust. YOLO optimiert eine Summe quadratischer Fehler. Die ist einfach zu optimieren, benötigt aber drei bewusst gesetzten Korrekturen, die typisch für praktische Verlustfunktionen sind:

1. Koordinatenfehler werden mit $\lambda_{\text{coord}} = 5$ hoch-, Konfidenzfehler leerer Zellen mit $\lambda_{\text{noobj}} = 0,5$ heruntergewichtet. Ohne das würden die vielen leeren Zellen die Konfidenzen gegen null drücken und den Gradienten der wenigen belegten Zellen überstimmen und das Training würde divergieren.
2. Breite und Höhe gehen als $\sqrt{w}$, $\sqrt{h}$ ein, damit derselbe Pixelfehler in kleinen Boxen mehr zählt als in großen.

3. Von den B Boxen einer Zelle ist nur diejenige mit der höchsten aktuellen IoU „zuständig“. Nur sie erhält Koordinaten- und Konfidenzgradienten. Die Prädiktoren spezialisieren sich dadurch auf Größen und Seitenverhältnisse.

Ergebnis und Grenzen. YOLO läuft mit 45 Bildern pro Sekunde bei 63,4 % mAP auf VOC 2007; Fast YOLO (9 Schichten) mit 155 fps bei 52,7 % und damit doppelt so genau wie jeder andere Echtzeitdetektor der Zeit (Abb. 13). Faster R-CNN war mit 73,2 % genauer, aber mit 7 fps sechsmal langsamer [3]. Die Fehleranalyse ist lehrreich: YOLO macht *mehr Lokalisierungsfehler* (19 % gegenüber 8,6 % bei Fast R-CNN). Die Boxen sitzen also ungenau, vor allem bei kleinen Objekten, aber *weniger als halb so viele Hintergrund-Fehlalarme* (4,75 % gegenüber 13,6 %), weil es das ganze Bild sieht und Kontext nutzen kann, statt isolierte Regionen zu bewerten. Zwei strukturelle Grenzen folgen direkt aus der Ausgabestruktur: Eine Zelle kann nur *eine* Klasse und $B = 2$ Boxen vorhersagen. An Gruppen kleiner Objekte (Vogelschwärme) wird hier gescheitert. Und die Boxregression aus groben 7×7 -Features begrenzt die Präzision. Spätere Versionen (YOLOv2 bis v8 und darüber hinaus) haben genau diese Punkte mittels Anker-Boxen, mehrere Skalen, Vorhersage auf feineren Feature Maps adressiert und halten den Ansatz bis heute im Echtzeit-Einsatz.

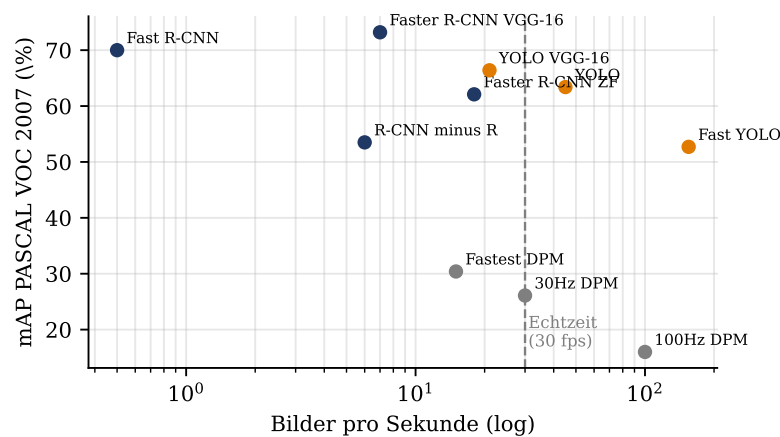


Abbildung 13: Genauigkeit gegen Geschwindigkeit auf PASCAL VOC 2007 (Zahlen aus [3, Tab. 1]). YOLO und Fast YOLO sind die einzigen Detektoren rechts der Echtzeitgrenze mit mAP über 50 %; die R-CNN-Familie ist genauer, aber ein bis zwei Größenordnungen langsamer.

Merke

Zwei Entwurfsmuster für Detektoren: **Zweistufig** (Faster R-CNN: erst Regionen vorschlagen, dann klassifizieren) ist genauer, besonders bei kleinen Objekten; **einstufig** (YOLO, SSD, RetinaNet: Gitter und direkte Regression) ist schneller und einfacher zu deployen. Die Wahl ist dieselbe Abwägung wie in Tab. 5: Latenzbudget und Objektgröße entscheiden. Beide Familien benutzen als Backbone ein Klassifikationsnetz, häufig ein ResNet. Das Trägermodell könnte also, mit anderem Kopf, ein Detektor werden.

7 Segmentierung (Überblick)

7.1 Aufgabe und Ausgabestruktur

Bei der *semantischen Segmentierung* wird jedes Pixel einer von C Klassen zugeordnet. Die Ausgabe hat dieselbe räumliche Größe wie das Bild, aber C Kanäle, die pro Pixel eine Klassenverteilung tragen; das Ergebnis lässt sich als eingefärbtes Bild darstellen (Straße, Fahrzeug, Fußgänger). Die *Instanzsegmentierung* unterscheidet zusätzlich einzelne Objekte derselben Klasse (Fußgänger 1, Fußgänger 2); sie kombiniert

Detektion mit einer Maske pro Box (Mask R-CNN). Der Verlust ist in beiden Fällen die pixelweise Cross-Entropy, oft ergänzt um Maße wie den Dice-Koeffizienten, die die IoU pro Klasse direkt optimieren.

Das Problem: Ein Klassifikations-CNN wirft räumliche Information gezielt weg und Pooling sowie Stride reduzieren 224×224 auf 7×7 . Für die Segmentierung muss die Auflösung am Ende aber wieder da sein.

7.2 Encoder-Decoder und Upsampling

Die Lösung ist eine Encoder-Decoder-Architektur [19]: Ein gewöhnlicher Backbone (Encoder) reduziert die Auflösung und erhöht die Kanalzahl. Ein Decoder kehrt das um und erhöht die Auflösung wieder auf die Bildgröße (Abb. 14). Für das Hochrechnen gibt es drei Bausteine [4]:

- **Unpooling (parameterfrei)**: jeder Wert wird in einen 2×2 -Block kopiert (Analogon zu Average-Pooling), oder er wird an die Position gesetzt, die beim Max-Pooling das Maximum hatte (*max-unpooling* mit gespeicherten Indizes).
- **Transponierte Faltung (gelernt)** (*fractionally strided convolution*): jeder Eingabewert wird mit dem Kernel multipliziert und in ein größeres Ausgabefeld gestreut, überlappende Beiträge werden summiert. Der Name kommt daher, dass die Operation in Matrixform die Transponierte der Stride-2-Faltung ist. Der Begriff „Deconvolution“ ist verbreitet, aber irreführend da es sich *nicht* um eine Umkehrung der Faltung handelt.
- **Bilineare Interpolation + Faltung**: hochskalieren mit einer festen Interpolation, dann eine normale Faltung um Schachbrettartefakte transponierter Faltungen zu vermeiden.

Ein Netz ohne vollverbundene Schichten, das nur aus Faltungen (und deren Transponierten) besteht, heißt *Fully Convolutional Network* (FCN). Es akzeptiert Bilder beliebiger Größe und gibt Karten derselben Größe zurück [19].

7.3 U-Net

Ein reiner Encoder-Decoder verliert feine Ortsinformation im Flaschenhals: Wo genau eine Objektkante lag, ist nach dem Downsampling auf 7×7 nur noch grob bekannt. U-Net [20] löst das mit *Skip-Verbindungen* zwischen Encoder- und Decoder-Stufen gleicher Auflösung: Die Feature Maps des Encoders werden mit den hochgerechneten Decoder-Maps konkateniert, sodass der Decoder auf hochaufgelöste Kanteninformation zugreifen kann, während er semantische Information aus dem Flaschenhals bekommt. Ursprünglich für biomedizinische Bilder mit wenigen Trainingsbeispielen entworfen, ist U-Net heute die Standardarchitektur für Segmentierung. In abgewandelter Form findet es sich außerdem als das Rückgrat von Diffusionsmodellen zur Bildgenerierung wieder. Die Skip-Verbindungen sind konzeptionell dieselbe Idee wie in ResNet: ein direkter Pfad, der Information (und Gradienten) an einer tiefen Verarbeitungskette vorbeiführt.

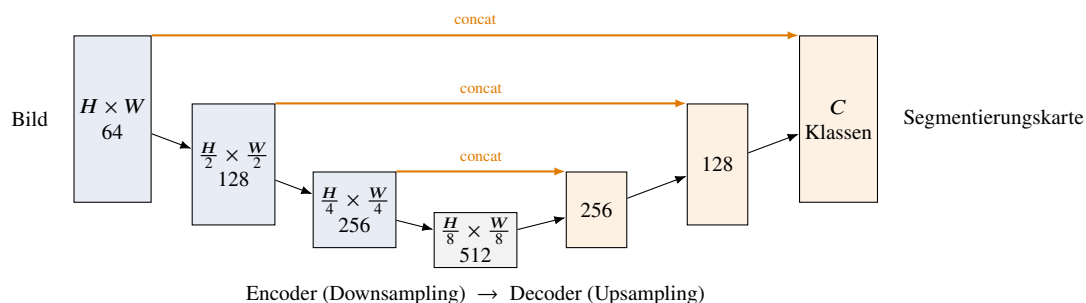


Abbildung 14: U-Net (schematisch): symmetrischer Encoder und Decoder; die Feature Maps jeder Encoder-Stufe werden per Skip-Verbindung mit der Decoder-Stufe gleicher Auflösung konkateniert. Am Ende reduziert eine 1×1 -Faltung auf C Klassenkanäle, gefolgt von einem pixelweisen Softmax.

Merke

Alle drei Aufgaben (Klassifikation, Detektion, Segmentierung) teilen sich den selben Backbone. Was sich ändert, ist der Kopf: ein Vektor (Klassifikation), ein Gitter von Boxen (Detektion) oder eine Karte in Bildgröße (Segmentierung). Wer den Backbone versteht und Transfer Learning beherrscht, kann jeden dieser Köpfe mit vergleichsweise wenig Daten aufsetzen. Das ist der praktische Grund, warum das Trägermodell ein Klassifikator ist: Er ist der einfachste Kopf und die Basis für alle anderen.

8 Fehlklassifikationen analysieren und ins Netz hineinsehen

Eine Validierungsgenauigkeit von 92 % sagt nicht, *welche* 8 % falsch sind. Genau hier entscheidet sich jedoch, ob ein Modell einsetzbar ist. Im folgenden wird behandelt, wie sich aus Fehlern Hypothesen ableiten lassen. Drei Werkzeuge in aufsteigender Tiefe werden dazu behandelt: Konfusionsmatrix, Beispielbilder, Blick in die Feature Maps.

8.1 Konfusionsmatrix und klassenweise Metriken

Die Konfusionsmatrix $\mathbf{M} \in \mathbb{N}^{C \times C}$ zählt, wie oft die wahre Klasse i als Klasse j vorhergesagt wurde (Abb. 15). Die Diagonale sind die Treffer; die Genauigkeit ist $\text{tr} \mathbf{M} / \sum_{ij} M_{ij}$. Interessant sind die *Off-Diagonal-Einträge*: Sie zeigen, ob die Fehler gleichverteilt sind (Rauschen, Modell zu schwach) oder sich auf wenige Paare konzentrieren (systematische Verwechslung, oft ein Datenproblem). Pro Klasse liest man ab:

$$\text{Precision}_j = \frac{M_{jj}}{\sum_i M_{ij}} \quad (\text{Wie viele der als } j \text{ Vorhergesagten sind } j?), \quad (7)$$

$$\text{Recall}_i = \frac{M_{ii}}{\sum_j M_{ij}} \quad (\text{Wie viele der wahren } i \text{ werden gefunden?}). \quad (8)$$

Bei unbalancierten Klassen ist die Genauigkeit irreführend. Dann sind der *Macro-F1* (Mittel der klassenweisen $F_1 = 2PR/(P + R)$) oder die *balanced accuracy* (Mittel der Recalls) als korrekte Kennzahlen zu wählen.

wahr	Kettensäge	92	1	3	2	2
	Kirche	2	94	1	1	2
	Kassettenrec.	9	1	86	3	1
	Golfball	1	0	2	95	2
	Fallschirm	2	4	1	3	90
		Kettensäge	Kirche	Kassettenrec.	Golfball	Fallschirm
		vorhergesagt				

Abbildung 15: Konfusionsmatrix des Trägermodells auf der Validierungsmenge (beispielhaft, 100 Bilder je Klasse). Die auffällige Verwechslung Kassettenrecorder → Kettensäge (9 Fälle) ist kein Zufall: Beide Klassen zeigen häufig graue, längliche Geräte mit Griffen vor unstrukturiertem Hintergrund.

8.2 Beispiele ansehen

Zahlen sind der Anfang, nicht das Ende. Für jedes auffällige Paar (i, j) zeige man die Bilder mit der höchsten Konfidenz für die *falsche* Klasse, also die „überzeugten“ Fehler. Typische Befunde und ihre Konsequenzen lauten:

- **Label-Fehler:** Das Bild ist tatsächlich Klasse j , das Label ist falsch. Bei kuratierten Teilmengen selten, in echten Projekten 1–5 % der Daten. Konsequenz: Daten bereinigen, nicht das Modell ändern.
- **Mehrdeutige Bilder:** Beide Klassen sind sichtbar (Golfball *und* Fallschirm). Konsequenz: Label-Schema prüfen; ggf. Multi-Label statt Multi-Class.
- **Kurzschluss über Kontext (*shortcut learning*):** Das Modell hat gelernt, dass Kirchen vor blauem Himmel stehen, und klassifiziert einen Fallschirm vor Himmel als Kirche. Konsequenz: Augmentierung, mehr Hintergrundvarianz, Erklärungsmethoden (unten) zur Bestätigung.
- **Verteilungsverschiebung:** Die Fehler häufen sich bei Bildern, die anders aussehen als das Training (Nachtaufnahmen, Nahaufnahmen, andere Kamera). Konsequenz (vorerst): dokumentieren, in die Validierung aufnehmen.
- **Zu wenig Kapazität / zu wenig Daten:** Fehler ohne erkennbares Muster, Trainingsgenauigkeit ebenfalls begrenzt. Konsequenz: mehr Schichten feinjustieren, größerer Backbone, mehr Daten.

In der Praxis

Analyseprotokoll für das Trägermodell. (1) Konfusionsmatrix und klassenweise Precision/Recall auf der Validierungsmenge; (2) Die drei häufigsten Verwechslungspaare benennen; (3) je Paar die fünf überzeugtesten Fehler anzeigen; (4) jeden Fehler einer der Kategorien oben zuordnen; (5) eine Maßnahme ableiten und ihre erwartete Wirkung *vor* dem nächsten Trainingslauf aufschreiben. Punkt 5 ist der Unterschied zwischen Analyse und Raten.

8.3 Was hat das Netz gelernt?

Die Filter der ersten Schicht lassen sich direkt als kleine Bilder anzeigen: Sie sind Gewichte auf $7 \times 7 \times 3$ Pixeln. Bei jedem auf natürlichen Bildern trainierten CNN zeigen sie orientierte Kanten, Farbübergänge und Gitter mit verblüffender Ähnlichkeit zu den *Gabor-Filtern*, mit denen die Neurowissenschaft die Antworten einfacher Zellen im visuellen Kortex modelliert [4]. Das belegt weniger, dass CNNs das Gehirn nachbilden, als dass diese Filter eine Eigenschaft der Bildstatistik sind. Das bedeutet, dass jedes System, welches natürliche Bilder verarbeitet, sie findet.

Tiefere Schichten lassen sich nicht direkt zeichnen, weil ihre Eingaben Filterantworten statt Pixel sind. Zwei indirekte Wege[21], [22]: (a) Man sucht im Validierungsdatensatz die Bildausschnitte, die eine Einheit am stärksten aktivieren oder (b) man optimiert per Gradientenaufstieg ein Eingabebild, das eine Einheit maximal aktiviert. Beide zeigen dieselbe Hierarchie: Kanten (Schicht 1), Texturen und einfache Formen (Schicht 2), Objektteile wie Räder oder Augen (Schicht 3), ganze Objekte (Schicht 5). Diese Hierarchie ist der empirische Grund, warum Transfer Learning funktioniert und warum die frühen Schichten eingefroren bleiben können (Abschn. 5).

Saliency Maps und Grad-CAM. Für die Fehleranalyse ist eine dritte Frage wichtiger: *Welche Bildregion* hat die Entscheidung getragen? Grad-CAM [23] berechnet für die Zielklasse c die Gradienten der Logit-Ausgabe $a^{(c)}$ nach den Feature Maps $A^{(k)}$ der letzten Faltungsschicht, mittelt sie pro Kanal zu Gewichten α_k und bildet die gewichtete Summe

$$\alpha_k = \frac{1}{M_k} \sum_{i,j} \frac{\partial a^{(c)}}{\partial A_{ij}^{(k)}}, \quad L^{(c)} = \text{ReLU} \left(\sum_k \alpha_k A^{(k)} \right), \quad (9)$$

die als Heatmap (7×7 bei ResNet, auf Bildgröße interpoliert) über das Bild gelegt wird. Die letzte Faltungsschicht ist die richtige Wahl, weil sie noch räumliche Auflösung hat, aber bereits die höchste semantische Ebene erreicht [4].

Adversariale Beispiele. Die Gradienten nach dem Eingabebild lassen sich auch missbrauchen: Mit $\mathbf{x}' = \mathbf{x} + \varepsilon \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, t))$ (*fast gradient sign method*) entsteht ein für Menschen unverändertes Bild, welches das Netz mit hoher Konfidenz falsch klassifiziert. Im bekannten Beispiel wird ein Panda (57,7 %) als Gibbon (99,3 %) Klassifiziert [4], [24]. Solche Störungen übertragen sich zwischen Modellen und funktionieren sogar physisch: Aufkleber auf Stoppschildern lassen CNNs ein Tempolimit erkennen. Daraus folgt, dass eine hohe Validierungsgenauigkeit kein Beleg für robuste Merkmale ist.

Literatur

- [1] I. Goodfellow, Y. Bengio und A. Courville, *Deep learning* (Adaptive computation and machine learning), en. Cambridge, Massachusetts: The MIT Press, 2016, ISBN: 978-0-262-03561-3.
- [2] K. He, X. Zhang, S. Ren und J. Sun, *Deep Residual Learning for Image Recognition*, en, arXiv:1512.03385 [cs.CV], Dez. 2015. DOI: [10.48550/arXiv.1512.03385](https://arxiv.org/abs/1512.03385). Adresse: <http://arxiv.org/abs/1512.03385>.
- [3] J. Redmon, S. Divvala, R. Girshick und A. Farhadi, *You Only Look Once: Unified, Real-Time Object Detection*, en, arXiv:1506.02640 [cs.CV], Mai 2016. DOI: [10.48550/arXiv.1506.02640](https://arxiv.org/abs/1506.02640). Adresse: <http://arxiv.org/abs/1506.02640>.
- [4] C. M. Bishop und H. Bishop, *Deep Learning: Foundations and Concepts*, en. Cham: Springer International Publishing, 2024, ISBN: 978-3-031-45467-7 978-3-031-45468-4. DOI: [10.1007/978-3-031-45468-4](https://link.springer.com/10.1007/978-3-031-45468-4). Adresse: <https://link.springer.com/10.1007/978-3-031-45468-4>.
- [5] K. P. Murphy, *Probabilistic machine learning: an introduction* (Adaptive computation and machine learning series), en. Cambridge, Massachusetts: The MIT Press, 2022, ISBN: 978-0-262-04682-4.
- [6] Y. Lecun, L. Bottou, Y. Bengio und P. Haffner, „Gradient-based learning applied to document recognition,“ *Proceedings of the IEEE*, Jg. 86, Nr. 11, S. 2278–2324, Nov. 1998, ISSN: 00189219. DOI: [10.1109/5.726791](https://ieeexplore.ieee.org/document/726791). Adresse: <http://ieeexplore.ieee.org/document/726791/>.
- [7] A. Krizhevsky, I. Sutskever und G. E. Hinton, „ImageNet classification with deep convolutional neural networks,“ en, *Advances in Neural Information Processing Systems*, S. 1097–1105, 2012, ISSN: 0001-0782, 1557-7317. DOI: [10.1145/3065386](https://dl.acm.org/doi/10.1145/3065386). Adresse: <https://dl.acm.org/doi/10.1145/3065386>.
- [8] K. Simonyan und A. Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, en, arXiv:1409.1556 [cs.CV], Apr. 2015. DOI: [10.48550/arXiv.1409.1556](https://arxiv.org/abs/1409.1556). Adresse: <http://arxiv.org/abs/1409.1556>.
- [9] C. Szegedy u. a., *Going Deeper with Convolutions*, en, arXiv:1409.4842 [cs.CV], Sep. 2014. DOI: [10.48550/arXiv.1409.4842](https://arxiv.org/abs/1409.4842). Adresse: <http://arxiv.org/abs/1409.4842>.
- [10] S. Ioffe und C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, en, arXiv:1502.03167 [cs.LG], März 2015. DOI: [10.48550/arXiv.1502.03167](https://arxiv.org/abs/1502.03167). Adresse: <http://arxiv.org/abs/1502.03167>.
- [11] M. Abu-zanona, S. Elaiwat, S. Younis, N. Innab und M. M. Kamruzzaman, „Classification of Palm Trees Diseases using Convolution Neural Network,“ en, *International Journal of Advanced Computer Science and Applications*, Jg. 13, Nr. 6, 2022, ISSN: 21565570, 2158107X. DOI: [10.14569/IJACSA.2022.01306111](https://thesai.org/Publications/ViewPaper?Volume=13&Issue=6&Code=IJACSA&SerialNo=111). Adresse: <http://thesai.org/Publications/ViewPaper?Volume=13&Issue=6&Code=IJACSA&SerialNo=111>.
- [12] L. Tychsen-Smith und L. Petersson, „Improving Object Localization with Fitness NMS and Bounded IoU Loss,“ en, in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Salt Lake City, UT: IEEE, Juni 2018, S. 6877–6885, ISBN: 978-1-5386-6420-9. DOI: [10.1109/CVPR.2018.00719](https://ieeexplore.ieee.org/document/8578817/). Adresse: <https://ieeexplore.ieee.org/document/8578817/>.
- [13] D. Desai u. a., „Model Quality in AI-based Bruise Detection: Rethinking IoU and Confidence Thresholds,“ en, 2025.

- [14] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus und Y. LeCun, *OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks*, en, arXiv:1312.6229 [cs.CV], Feb. 2014. doi: [10.48550/arXiv.1312.6229](https://doi.org/10.48550/arXiv.1312.6229). Adresse: <http://arxiv.org/abs/1312.6229>.
- [15] R. Girshick, J. Donahue, T. Darrell und J. Malik, „Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,“ en, in *2014 IEEE Conference on Computer Vision and Pattern Recognition*, Columbus, OH, USA: IEEE, Juni 2014, S. 580–587, ISBN: 978-1-4799-5118-5. doi: [10.1109/CVPR.2014.81](https://doi.org/10.1109/CVPR.2014.81). Adresse: <http://ieeexplore.ieee.org/document/6909475/>.
- [16] R. Girshick, „Fast R-CNN,“ in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [17] S. Ren, K. He, R. Girshick und J. Sun, *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*, en, arXiv:1506.01497 [cs.CV], Jan. 2016. doi: [10.48550/arXiv.1506.01497](https://doi.org/10.48550/arXiv.1506.01497). Adresse: <http://arxiv.org/abs/1506.01497>.
- [18] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn und A. Zisserman, „The Pascal Visual Object Classes (VOC) Challenge,“ en, *International Journal of Computer Vision*, Jg. 88, Nr. 2, S. 303–338, Juni 2010, ISSN: 0920-5691, 1573-1405. doi: [10.1007/s11263-009-0275-4](https://doi.org/10.1007/s11263-009-0275-4). Adresse: <http://link.springer.com/10.1007/s11263-009-0275-4>.
- [19] J. Long, E. Shelhamer und T. Darrell, „Fully convolutional networks for semantic segmentation,“ in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA: IEEE, Juni 2015, S. 3431–3440, ISBN: 978-1-4673-6964-0. doi: [10.1109/CVPR.2015.7298965](https://doi.org/10.1109/CVPR.2015.7298965). Adresse: <http://ieeexplore.ieee.org/document/7298965/>.
- [20] O. Ronneberger, P. Fischer und T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, en, arXiv:1505.04597 [cs.CV], Mai 2015. doi: [10.48550/arXiv.1505.04597](https://doi.org/10.48550/arXiv.1505.04597). Adresse: <http://arxiv.org/abs/1505.04597>.
- [21] M. D. Zeiler und R. Fergus, „Visualizing and Understanding Convolutional Networks,“ en, in *Computer Vision – ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele und T. Tuytelaars, Hrsg., Bd. 8689, Series Title: Lecture Notes in Computer Science, Cham: Springer International Publishing, 2014, S. 818–833, ISBN: 978-3-319-10589-5 978-3-319-10590-1. doi: [10.1007/978-3-319-10590-1_53](https://doi.org/10.1007/978-3-319-10590-1_53). Adresse: http://link.springer.com/10.1007/978-3-319-10590-1_53.
- [22] J. Yosinski, J. Clune, A. Nguyen, T. Fuchs und H. Lipson, *Understanding Neural Networks Through Deep Visualization*, en, arXiv:1506.06579 [cs.CV], Juni 2015. doi: [10.48550/arXiv.1506.06579](https://doi.org/10.48550/arXiv.1506.06579). Adresse: <http://arxiv.org/abs/1506.06579>.
- [23] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh und D. Batra, „Grad-CAM: Visual Explanations From Deep Networks via Gradient-Based Localization,“ en, in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.
- [24] I. J. Goodfellow, J. Shlens und C. Szegedy, *Explaining and Harnessing Adversarial Examples*, en, arXiv:1412.6572 [stat.ML], März 2015. doi: [10.48550/arXiv.1412.6572](https://doi.org/10.48550/arXiv.1412.6572). Adresse: <http://arxiv.org/abs/1412.6572>.